

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«УФИМСКИЙ ГОСУДАРСТВЕННЫЙ НЕФТЯНОЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Кафедра вычислительной техники и инженерной кибернетики

**Программирование на Си в Unix с использованием примитивов
синхронизации**

**Учебно – методическое пособие к лабораторным работам
по дисциплине «Операционные системы»**

**Уфа
Издательство УГНТУ
2025**

Рассмотрены особенности программирования на языке Си в Unix. Даны практические рекомендации для реализации иерархии родственных процессов. Изложены способы передачи данных между процессами и с помощью программных каналов. Рассмотрена синхронизация процессов с помощью очередей сообщений. Приведен пример взаимодействия процессов с использованием разделяемых областей памяти и семафоров. Даны рекомендации по многопоточному программированию и синхронизации работы потоков с помощью мьютексов и условных переменных.

Составители: Гиниятуллин В.М., канд. техн. наук, доцент кафедры ВТИК
Шаяхметов Э.В., аспирант группы А0946-23-01

Рецензент: Тулупова О.П., канд. техн. наук, доцент кафедры ВТИК
Филиппов В.Н., канд. техн. наук, доцент кафедры ВТИК

Лабораторная работа «Счетные ресурсы».

Сравнение вычислительной мощности, предоставляемой операционными системами

Скорость математических вычислений с плавающей запятой определяется в первую очередь аппаратными средствами, однако большое значение имеет поддержка операционной системой (ОС) возможностей аппаратуры и рациональный алгоритм программы.

Для сравнения предоставляемой операционными системами вычислительной мощности необходимо написать программу циклического вычисления некоторой функции, например, такой:

```
for(i = 0; i < n; i++)  
    result = i * 3 + sin(i) - sqrt(1 / 15);
```

и оценить время её исполнения.

Субъективное время пользователя измеряется в секундах, соответственно необходимо подбирать такое количество итераций, чтобы время работы составляло примерно 10 секунд. На практике результатом эксперимента называют среднее арифметическое значений не менее чем трех опытов.

Время, затрачиваемое на собственно вычисления и вывод результатов, может значительно различаться, в таких случаях, допустимо уменьшать количество итераций, указывая коэффициент изменения (кратный десяти) в таблице результатов. Кроме того, замеры времени необходимо сделать дважды, с неявной записью и принудительно сбрасывая все буферизированные данные на диск.

Кроссплатформенный способ принудительного сброса буферизованных данных на диск реализован в системном вызове `fsync()`, во всех операционных системах имеются соответствующие системные вызовы (UNIX) и API функции (Windows).

После определения временных затрат на «чистые» вычисления в цикл следует добавить вывод на экран и в файл, отдельно и совместно. Результаты замеров оформляются в таблице следующего вида:

Таблица 1

Время исполнения программ

	fpu, с.	fpu+video, с.	fpu+hdd, с.	fpu+video+hdd, с.
Windows				
Windows с fsync()	-			
UNIX				
UNIX с fsync()	-			

Затем необходимо повторить все опыты в двух и более окнах командной строки, на фоне `gnome-system-monitor` (UNIX) и `task manager` (Windows). Для всех опытов зафиксировать в отчете количество/процент загруженности ядер и изменение времени работы программ. Сравнить содержимое выходных файлов при одновременной записи в один файл (некорректный вариант) и разные файлы (рекомендованный способ).

Выводы должны содержать собственные соображения о применимости каждой ОС.

Лабораторная работа «Память девичья».
Сравнение объемов памяти предоставляемых,
операционными системами

Надежная работа оперативной памяти компьютера чрезвычайно важна. Подсистема памяти, с одной стороны, должна обеспечить программисту доступ к максимальному объему памяти, а с другой иметь надежную защиту от некорректных действий. В данной работе специально используются некорректные приемы работы с памятью.

Для сравнения размера динамической памяти, предоставляемой ОС, необходимо написать программу преднамеренно захватывающей памяти больше, чем её есть в компьютере. Например, если в компьютере установлено 2 Гб физической памяти, тогда двумерный массив из 23171 строки и 23171 столбца float – ов будет занимать $23\,171 * 23\,171 * 4 = 2\,174\,580\,964$ байта, что на 98 кБ больше 2 Гб.

Стандартные средства динамического распределения динамического памяти: функция – malloc() и оператор new, при неудачном завершении должны возвращать код ошибки (NULL). Однако последние версии компилятора gcc/g++ используют так называемое «отложенное выделение памяти», собственно захват памяти производится не malloc()/new, а в момент её инициализации. При этом инициализация одного элемента массива приводит к выделению целой страницы памяти (обычно 4 кБ).

Инициализировать можно не все элементы массива, а выводя количество итераций цикла, тогда можно будет оценить объем доступной памяти. Сравнить поведение двух вариантов программ: инициализация в цикле захвата памяти и отдельные циклы захвата и инициализации. Одновременно запустить программы в двух окнах на фоне gnome-system-monitor (UNIX) и task manager (Windows). Для всех опытов зафиксировать в отчете реакцию операционной системы на преднамеренное переполнение памяти.

Для сравнения размера сегмента стека, предоставляемой ОС, необходимо написать программу бесконечного рекурсивного вызова собственной функции, выводящей количество итераций рекурсии. Одним из параметров функции, следует сделать массив «по значению».

Затем необходимо повторить все опыты в двух и более окнах командной строки, на фоне `gnome-system-monitor` (UNIX) и `task manager` (Windows). Для всех опытов зафиксировать в отчете реакцию операционной системы на преднамеренное нарушение при захвате памяти и переполнении сегмента стека.

Выводы должны содержать собственные соображения о применимости каждой ОС.

Лабораторная работа «Наследники».

Создание дочерних процессов

Все процессы в UNIX – системах, кроме самого первого, который создается при первоначальной загрузке системы, создаются с помощью системного вызова `fork()`. После завершения вызова `fork()` и родительский и порожденный процессы продолжают свое выполнение одновременно.

Порожденный процесс является почти полной копией родительского, в частности он наследует следующие атрибуты процесса:

- реальный идентификатор владельца (rUID) – идентификатор пользователя, который создал родительский процесс. С помощью этого атрибута ядро операционной системы следит за тем, кто и какие процессы создает в системе;
- реальный идентификатор группы (rGID) – идентификатор группы, к которой принадлежит пользователь, создавший родительский процесс;
- эффективный идентификатор пользователя (eUID) – обычно совпадает с (rUID);
- эффективный идентификатор группы (eGID) – обычно совпадает с (rGID);
- текущий каталог – номер индексного дескриптора на текущий каталог;
- корневой каталог – номер индексного дескриптора на корневой каталог;
- параметры обработки сигналов;
- права доступа к файлам;
- значение приоритета процесса (nice).

Дочерний и родительский процессы различаются:

- идентификатором процесса (PID) – целочисленный идентификатор, уникальный для процесса во всей операционной системе;
- идентификатором родительского процесса (PPID);
- блокировками файлов.

При удачном завершении системного вызова `fork()` родительский процесс может посредством системного вызова `wait()` приостановить свое выполнение до

завершения порожденного процесса или продолжать свое выполнение независимо от порожденного процесса.

Процесс завершает свою работу при помощи системного вызова `exit()`. Аргументом этого вызова является код завершения процесса, обычно ноль символизирует удачное завершение, а отрицательные значения сообщают о неудаче. Хорошим стилем программирования считается, расшифровка причин ошибки. Код завершения процесса сообщается родителю с помощью системного вызова `wait()`.

Системные вызовы семейства `exec()` перезагружают текущее адресное пространство другим исполняемым модулем, т.е. запускают новую программу.

Системные вызовы `fork()` и `exec()`, как правило, используются совместно. Например, `shell` выполняет каждую команду пользователя путем вызова `fork()` и `exec()` и каждая затребованная команда выполняется в своем собственном процессе.

Прототип системного вызова `fork()`:

```
pid_t fork(void);
```

Функция `fork()` не принимает аргументов и возвращает значение типа `pid_t`. Этот вызов при:

- удачном завершении, создает порожденный процесс. Родительскому процессу он возвращает идентификатор (PID) потомка, в порожденный процесс возвращает 0;
- неудачном завершении, порожденный процесс не создаёт. Родительскому процессу он возвращает код ошибки.

Следует подчеркнуть, что сегменты кода, данных, стека у родительского и дочернего процессов одинаковы, более того оба процесса продолжают свое исполнение со следующей за `fork()` строкой кода, но каждый в своем адресном пространстве.

Прототип системного вызова `exit()`:

```
void exit(int status);
```


Аргумент функции `exit()` – это целочисленный статус завершения процесса, родительский процесс может получить только младшие 8 бит этого кода. Ненулевые значения кода завершения, обычно, используются для сигнализации об ошибке. Функция `exit()` никогда не выполняется неудачно, поэтому для нее нет возвращаемого значения.

По завершении работы процесса, выделенные ему ресурсы, такие как сегменты кода, данных, стека и т.п. возвращаются под управление ОС, а вот запись в таблице процессов остается нетронутой, говорят, что процесс перешел в состояние «зомби». С помощью системного вызова `wait()` можно получить статус завершения процесса, а также освободить его строку в таблице процессов, т.е. окончательно удалить процесс из системы.

Прототип системного вызова `wait()`:

```
pid_t wait(int *status);
```

Аргумент функции `wait()` – это статус завершения дочернего процесса, а возвращаемое значение – это PID завершившегося дочернего процесса.

Если порожденный процесс ещё не завершился, то родительский процесс будет «спать внутри» `wait()` в ожидании завершения потомка. Если порожденных процессов несколько, то родительский процесс «проснется» по завершении любого из них. При отсутствии порожденных процессов `wait()` немедленно завершается с отрицательным кодом ошибки. Родительский процесс не обязан ожидать завершения работы потомка и может завершить свою работу раньше, в этом случае говорят, что потомок стал «сиротой». Всех сирот «усыновляет» специализированный процесс, обычно `init`, этот процесс создается первым при запуске ОС, особым образом, и завершается последним, т.е. гарантируется, что он всегда присутствует в системе.

Системные вызовы семейства `exec()` подменяют текущий контекст процесса, т.е. запускают другой исполняемый модуль.

Прототипы системных вызовов `exec()`:

```
int execl(const char *path, const char *arg, ...);
```

```

int execlp(const char *file, const char *arg, ...);
int execl(const char *path, const char *arg, ...,
    char *const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[],
    char *const envp[]);

```

Аргументы системных вызовов `exec()` – это путь или имя запускаемого файла, затем следуют аргументы командной строки или массив строк с аргументами командной строки, кроме того может указываться массив строк окружения. При удачном завершении `exec()` ничего не возвращает, в противном случае возвращается -1, переменная `errno` содержит код ошибки.

Уточним, `exec()` записан в тексте родительского процесса, а выполняется в адресном пространстве потомка, следующей за `exec()` командой будет функция `main()` дочернего процесса.

Родительский процесс получает PID потомка с помощью системных вызовов `fork()` и `wait()`. Дочерний процесс может получить собственный PID и PID родителя с помощью системных вызовов `getpid()` и `getppid()`.

Прототипы системных вызовов `getpid()` и `getppid()`:

```

pid_t getpid(void);
pid_t getppid(void);

```

Функции `getpid()` и `getppid()` аргументов не имеют и возвращают идентификаторы текущего и родительского процессов соответственно. При необходимости можно получить значения реальных и/или эффективных идентификаторов пользователя и/или группы.

Каждый процесс может пребывать в двух фазах: «системной» или «ядерной» (внутри какого – либо системного вызова – его выполняет ядро ОС по запросу процесса) и «пользовательской» (внутри кода самого процесса). Время затраченное процессом в каждой фазе, может быть измерено системным вызовом

times(), кроме того этот вызов позволяет узнать суммарное время, затраченное всеми потомками процесса.

Прототип системного вызова times():

```
struct tms {
    clock_t tms_utime; /* user time */
    clock_t tms_stime; /* system time */
    clock_t tms_cutime; /* user time of children */
    clock_t tms_cstime; /* system time of children */
};

clock_t times(struct tms *buf);
```

Функция times() возвращает астрономическое время системы и заполняет структуру tms, которая содержит следующие поля:

- tms_utime – время, затраченное процессом в пользовательской фазе;
- tms_stime – время, затраченное процессом в ядерной фазе;
- tms_cutime – сумма времен, затраченных потомками процесса в пользовательской фазе;
- tms_cstime – сумма времен, затраченных потомками процесса в ядерной фазе.

На практике рекомендуется использовать не абсолютные значения времен, а разницы от двух вызовов times(). Все времена измеряются в «тиках», размерность «тика» в секундах не стандартизована и в разных ОС может различаться. Размерность «тика» можно найти в документации, а можно определить экспериментально. При неудачном завершении возвращается -1, переменная errno содержит код ошибки.

В приведенном ниже примере, исполняемый файл father создает трех потомков, и «дети» и «папа» выводят на экран свои и родительские идентификаторы и временные характеристики свои и потомков.

```
//son.cpp
int main(void)
{ int i, ut, st, cu, cs;
  double res;
  FILE *fOut;
  struct tms start_time, end_time;
  clock_t real_time;

  fOut = fopen("text.txt", "w");
  real_time = times(& start_time);
  for(i = 0; i < 500000; i++)
  { res = i*3 + sin(i) - sqrt(i/15);
    fprintf(fOut, "%e, ", res);
  }
  real_time = times(&end_time);
  ut = end_time.tms_utime - start_time.tms_utime;
  st = end_time.tms_stime - start_time.tms_stime;
  cu = end_time.tms_cutime - start_time.tms_cutime;
  cs = end_time.tms_cstime - start_time.tms_cstime;
  printf("sonPid=%d, ppid=%d, ut=%d,st=%d,cu=%d,cs=%d\n",
         getpid(), getppid(), ut, st, cu, cs);
  fclose(fOut);
  exit(0);
}
```

```

//father.cpp
int main(void)
{ int i, ut, st, cu, cs, status;
  double res;
  FILE *fOut;
  struct tms start_time, end_time;
  clock_t real_time;

  fOut = fopen("text1.txt", "w");
  real_time = times(& start_time);
  for(i = 0; i < 500000; i++)
  { res = i*3 + sin(i) - sqrt(i/15);
    fprintf(fOut, "%e, ", res);
  }
  for(i = 0; i < 3; i++)
  { if(fork() == 0)
      execl("./son", "son", NULL);
    else
      wait(&status);
  }
  real_time = times(&end_time);
  ut = end_time.tms_utime - start_time.tms_utime;
  st = end_time.tms_stime - start_time.tms_stime;
  cu = end_time.tms_cutime - start_time.tms_cutime;
  cs = end_time.tms_cstime - start_time.tms_cstime;
  printf("fatherPid=%d,ppid=%d,ut=%d,st=%d,cu=%d,cs=%d\n",
        getpid(), getppid(), ut, st, cu, cs);
  fclose(fOut);
  exit(0);
}

```

Построить генеалогическое дерево из 4-х поколений процессов. Количество процессов в каждом поколении выбирается из таблицы 4. Во 2-м поколении должны быть А процессов, в 3-м поколении В, а в 4-м С. Процессы должны существовать примерно 1 секунду и выводить на экран собственные и родительские идентификаторы и временные характеристики свои и потомков. Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов, кроме того продемонстрировать создание процессов сирот и зомби.

В отчете необходимо привести графическое представление генеалогического дерева.

Лабораторная работа «Трубы».

Использование программных каналов

Одним из штатных методов обмена данными между процессами являются программные каналы. Для чтения/записи в программный канал можно использовать библиотечные вызовы `read()/write()`. Указатель записи/чтения у программных каналов разделен и зафиксирован, запись производится только в начало, а чтение возможно только из хвоста канала, при этом данные изымаются из программного канала, а не копируются.

Неименованный программный канал создается системным вызовом `pipe()` и предназначен для однонаправленной передачи данных между родственными процессами. Например, так – родительский процесс с помощью `pipe()` создаёт программный канал, затем с помощью `fork()` создаёт дочерний процесс. Потомок наследует от родителя все открытые дескрипторы и, следовательно, имеет доступ к программному каналу, остаётся определить, какой из процессов будет писать, а какой читать. По завершении работы обоих процессов программный канал автоматически уничтожается. Прямой перевод слова `pipe` породил жаргонный термин «труба».

Прототип системного вызова `pipe()`:

```
int pipe(int pipeId[2]);
```

При удачном завершении возвращается 0, при неудачном -1, переменная `errno` содержит код ошибки. Аргумент функции, после удачного вызова, будет содержать дескрипторы открытого канала: `pipeId[0]` на чтение, а `pipeId[1]` на запись.

Для выполнения следующей команды в терминале:

```
ls > text.txt
```

shell неявным образом создаёт неименованный программный канал и два процесса, один из них читает содержимое текущего каталога, второй записывает

это содержимое в текстовый файл, собственно данные будут передаваться через программный канал.

Подчеркнем, неименованный программный канал не имеет индексного дескриптора и, следовательно, файлом не является.

Именованный программный канал создаётся системным вызовом `mkfifo()` или более общим вызовом `mknode()`. Именованные программные каналы – это файлы с особым поведением, они носят название FIFO – файлы и их можно увидеть в списке содержимого каталога.

В большинстве ОС программные каналы передают данные только в одном направлении, поэтому каждый программный канал должен быть открыт не менее чем двумя разными процессами, на запись в начало и на чтение из хвоста канала. Программный канал открывается библиотечным вызовом `open()`, в режиме «только чтение» или «только запись». Процесс первым открывший канал на чтение/запись блокируется до момента открытия канала процессом – «партнером».

Попытка чтения из пустого канала приводит к блокировке «читателя» до тех пор, пока в канале не появятся данные, либо канал не будет удален. FIFO – файлы, как и обычные файлы, имеют ограничения на размер, поэтому попытка записи в переполненный канал приведет к блокировке «писателя». Ситуаций с блокировкой на чтении/записи следует избегать. Количество процессов, подключенных к каналу на запись неограниченно, а вот наличие нескольких «читателей» недопустимо.

Удалить FIFO – файлы можно с помощью системного вызова `unlink()`, это указание для ОС уничтожить любой файловый объект, а не только FIFO – файл, после того как работу с этим объектом завершит последний процесс.

Прототип системного вызова `mkfifo ()`:

```
int mkfifo(const char *pathname, mode_t mode);
```


Первый аргумент путь/имя, второй режим создания, в частности права доступа. При удачном завершении возвращается 0, при неудачном -1, переменная errno содержит код ошибки.

Прототип системного вызова unlink():

```
int unlink(const char *pathname);
```

Аргументом является имя путь/файла, при удачном завершении возвращается 0, при неудачном -1, переменная errno содержит код ошибки.

В приведенном ниже примере процесс creator создаёт FIFO – файл, двух потомков, ожидает их завершения и удаляет канал. Процесс client1 читает из текстового файла строку, добавляет к ней свою метку, открывает программный канал на запись и записывает в канал модифицированную строку. Процесс client2 открывает программный канал на чтение, читает его содержимое, модифицирует и выводит на экран.

```
//creator.cpp
#define fifo "fifo"
int main(void)
{ int status;
  if(mkfifo(fifo, 0606) < 0)
  { printf("don't create fifo\n");
    exit(-1);
  }
  if(fork() == 0)
    execl("./client1", "client1", NULL);
  if(fork() == 0)
    execl("./client2", "client2", NULL);
  while(wait(&status)>0);
  unlink(fifo);
  printf("\n creatorPid= %d\n", getpid());
  exit(0);
}
```

```
//client1.cpp
#define fifo "fifo"
int main(void)
{ int pId1;
  FILE *fOut;
  char buff[100];
  fOut = fopen("text.txt", "r");
  fgets(buff, 13, fOut);
  strcat(buff, "+client1");
  pId1 = open(fifo, O_WRONLY);
  write(pId1, buff, 99);
  printf("client1=%d, pPid=%d, in file str=%s\n",
        getpid(), getppid(), buff);
  close(pId1);
  fclose(fOut);
  exit(0);
}

//client2.cpp
#define fifo "fifo"
int main(void)
{ int pId1;
  char buff[100];
  pId1 = open(fifo, O_RDONLY);
  read(pId1, buff, 99);
  strcat(buff, "+client2");
  printf("client2=%d, ppid=%d, in pipe str=%s\n",
        getpid(), getppid(), buff);
  close(pId1);
  exit(0);
}
```

Согласно номеру, в списке группы выбрать себе схему из приложения 2. Кружки на схеме символизируют процессы, черточки между ними – каналы, стрелки – входные/выходные файлы. Для каждого канала выбрать направление передачи данных, избегая при этом циклов. Один/несколько процессов читают строки из текстового файла и пересылают их по каналам другим процессам. Каждый из процессов добавляет к строке свою метку и отправляет её дальше. Последние процессы в цепочке записывают полученную строку в файл на диск.

Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов. Корректная реализация передачи данных через FIFO – файлы, это весьма кропотливая работа, поэтому рекомендуется использовать системы контроля версий.

Лабораторная работа «На деревню дедушке».

Использование очередей сообщений

Одним из штатных методов синхронизации работы процессов являются очереди сообщений, кроме того они относятся к механизмам межпроцессных взаимодействий Inter Process Connection (IPC).

Очередь сообщений – это область доступная для чтения/записи нескольким процессам, даже запущенным от имени разных пользователей. В адресном пространстве ядра имеется таблица очередей сообщений, в которой отслеживаются все очереди сообщений, созданные в ОС. Одна строка этой таблицы соответствует одной очереди сообщений, кроме прочих, в таблице имеются следующие столбцы:

- Ключ, целое положительное число, присвоенное очереди программистом, который его создал. Другие процессы могут, указывая этот ключ, «открывать» очередь и получать дескриптор для доступа к ней.
- UID и GID пользователя от имени, которого запущен процесс, создавший очередь сообщений. Процесс эффективный UID которого совпадает с UID пользователя – создателя очереди сообщений, имеет право удалять очередь и изменять параметры управления ею.
- UID и GID назначенного владельца. Обычно эти идентификаторы совпадают с идентификаторами создателя очереди, но создатель может изменить их на идентификаторы другого пользователя.
- Права доступа к очереди для чтения/записи по категориям «владелец», «группа» и «прочие».
- Время и идентификатор процесса, который последним передал сообщение в очередь.
- Время и идентификатор процесса, который последним прочитал сообщение из очереди.
- Указатель на связанный список сообщений, находящихся в очереди.

Когда процесс передает сообщение в очередь, ядро ОС создаёт для него новую запись и помещает её в конец связанного списка сообщений данной очереди. В каждой такой записи указывается целочисленный тип сообщения, размер сообщения в байтах, и указатель на другую область ядра, где фактически находятся сообщения. Ядро копирует данные, содержащиеся в сообщении из адресного пространства процесса – отправителя в область данных ядра, чтобы процесс – отправитель мог продолжить свою работу или даже завершиться, а сообщение осталось доступным для чтения другими процессами.

Когда процесс читает сообщение из очереди, ядро копирует относящиеся к нему данные в адресное пространство процесса и удаляет сообщение, т.е. сообщение изымается из очереди. Процесс может прочитать сообщение из очереди следующими способами:

- изъять одно самое старое сообщение указанного типа;
- изъять одно самое старое сообщение любого типа;
- изъять одно самое старое сообщение тип, которого находится в заданном диапазоне.

Если в очереди нет сообщений, подходящего типа, то процесс – «читатель» будет заблокирован до тех пор, пока в очереди не появится нужное сообщение.

Для работы с очередями сообщений определены следующие системные вызовы:

- `msgget()` создание/открытие очереди сообщений;
- `msgsnd()` отправляет сообщение в очередь сообщений;
- `msgrcv()` получение сообщения из очереди сообщений;
- `msgctl()` удаление очереди сообщений или получение/изменение управляющих параметров.

Уникальный ключ объекта IPC может быть сформирован системным вызовом `ftok()`, его прототип:

```
key_t ftok(const char *pathname, int id);
```

Первый аргумент путь/имя существующего файлового объекта. Второй аргумент модификатор ключа, если необходимо получить несколько ключей, то каждый следующий `ftok()` вызывается с другим модификатором. При удачном завершении возвращается положительный целочисленный ключ, при неудачном -1, переменная `errno` содержит код ошибки.

Очередь сообщений — это устойчивый объект, т.е. при завершении процесса создателя она остается в пространстве ядра, и другие процессы могут продолжать с ней работать. Для удаления области необходимо использовать в явном виде системный вызов `msgctl()`.

Прототип системного вызова `msgget()`:

```
int msgget(key_t key, int flag);
```

Первый аргумент `key` — положительный целочисленный ключ, необходимый программисту для обеспечения доступа к очереди из разных программ, для его формирования рекомендуется использовать `ftok()`. Если же значением `key` является макрос `IPC_PRIVATE`, то будет создана новая очередь сообщений, доступная только вызывающему процессу. Последний аргумент `flag` — это комбинация режима открытия и прав доступа, в таблице 2 приведена реакция системы на различные варианты содержимого `flag`.

Таблица 2

Режимы открытия объекта IPC

Значение <code>msgflg</code>	Объект не существует	Объект уже создан
0	ошибка, <code>errno = ENOENT</code>	открытие объекта
<code>IPC_CREAT</code>	создание нового объекта	открытие объекта
<code>IPC_CREAT IPC_EXCL</code>	создание нового объекта	ошибка, <code>errno = EEXIST</code>

При удачном завершении возвращается положительный целочисленный дескриптор, который необходим другим функциям семейства `msg`, при неудачном завершении -1, переменная `errno` содержит код ошибки.

Прототип системного вызова `msgsnd()`:

```
int msgsnd(int msqid, const void *msgp, size_t msgsz,
           int flag);
```

Первый аргумент функции это дескриптор очереди сообщений, который вернул вызов `msgget()`. Вторым аргументом — это указатель на объект, который содержит тип и собственно сообщение. Для объявления объекта можно использовать следующий тип данных:

```
struct mbuf
{ long mtype;           //тип сообщения
  char mtext[MSGMAX];   //буфер для текста сообщения
};
```

размер текстового буфера и его тип определяются программистом, в частности это поле структуры вообще может отсутствовать, а вот поле типа сообщения должно быть обязательно. Значение аргумента `msgsz` — это размер (в байтах) поля `mtext`. Аргумент `flag` в большинстве случаев принимает значение 0, если его значение равно `IPC_NOWAIT`, то выполняется неблокирующий вызов.

При удачном завершении возвращается 0, при неудачном -1, переменная `errno` содержит код ошибки.

Прототип системного вызова `msgrcv()`:

```
size_t msgrcv(int msqid, void *msgp, size_t msgsz,
              long msgtype, int flag);
```

Первый аргумент функции это дескриптор очереди сообщений, который вернул вызов `msgget()`. Вторым аргументом — это указатель на объект, куда будет помещено прочитанное сообщение. Значение аргумента `msgsz` — это максимальный размер (в байтах) сообщения, которое может быть принято этим вызовом, если сообщение длиннее, то возвращается код ошибки. Тип сообщения, подлежащего приему, кодируется аргументом `msgtype`, он может принимать значения:

- положительно целое, тогда из очереди будет изъято одно самое старое сообщение указанного типа;

- 0, тогда из очереди будет изъято одно самое старое сообщение любого типа;
- отрицательное целое, тогда из очереди будет изъято одно самое старое сообщение тип, которого находится в диапазоне от единицы до указанного значения.

Если аргумент `flag` принимает значение 0, то при отсутствии в очереди нужного сообщения процесс будет заблокирован внутри системного вызова `msgrcv()`. Системный вызов `msgrcv()` используется для синхронизации работы процессов потому, что он, в отличие от `msgsnd()`, обладает свойством атомарности. Если аргумент `flag` принимает значение `IPC_NOWAIT`, то при отсутствии в очереди нужного сообщения вызов вернет код ошибки. Кроме того, может быть установлен флаг `MSG_NOERROR`, тогда вызов можно читать сообщения, размер которых больше чем `msgsz`, лишние байты будут потеряны.

Функция возвращает количество прочитанных байт, -1 при неудачном завершении, переменная `errno` содержит код ошибки.

Прототип системного вызова `msgctl()`:

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

Первый аргумент функции это дескриптор очереди сообщений, который вернул вызов `msgget()`. Аргумент `cmd` может принимать значения:

- `IPC_STAT`, тогда вызов будет копировать управляющие параметры очереди в объект по адресу `buf`;
- `IPC_SET`, тогда вызов будет изменять управляющие параметры очереди на значения находящиеся в объекте по адресу `buf`;
- `IPC_RMID`, тогда очередь сообщений будет удалена из системы, а указателю `buf` можно присвоить значение `NULL`.

При удачном завершении возвращается 0, при неудачном -1, переменная `errno` содержит код ошибки.

В приведенном примере процесс `proba1` создаёт очередь сообщений и записывает в неё два сообщения разных типов. Файл `proba2` читает сообщения из очереди, а `proba3` удаляет очередь.


```
//proba1.cpp
struct mbuf
{ long mtype;
  char mtext[100];
}mobj={15, "Hello"}, mobj16={16, "By"};
int main()
{ int fd, key;
  key = ftok("proba1", 1);
  fd = msgget(key, IPC_CREAT|0606);
  msgsnd(fd, &mobj, strlen(mobj.mtext)+1, 0);
  msgsnd(fd, &mobj16, strlen(mobj16.mtext)+1, 0);
  exit(0);
}
```

```
//proba2.cpp
struct mbuf2
{ long mtype2;
  char mtext2[100];
}mobj2;
int main()
{ int fd2, i2, key2;
  key2 = ftok("proba1", 1);
  fd2 = msgget(key2, IPC_CREAT|0606);
  for(i = 0; i < 7; i++)
  { if(msgrcv(fd, &mobj, 101, -18, 0) > 0)
    printf("%s\n", mobj2.mtext2);
    else
    { printf("not queue\n");
      exit(0);
    } } }
```

```
//proba3.cpp
int main()
{ int fd, i, key3;
  key3 = ftok("proba1", 1);
  fd = msgget(key3, 0);
  if(i < 0)
    printf("dont remove msg\n");
  else
    exit(0);
  i = msgctl(fd, IPC_RMID, 0);
  printf("remove msg\n");
  exit(0);
}
```

В одну очередь сообщений А (таблица 4) серверов посылают сообщения, каждый своего типа. Первый сервер отправляет В сообщений, второй – С сообщений, остальные сервера отправляют произвольное количество сообщений. Один из клиентов получает все сообщения 3-го типа, второй клиент получает 5 сообщений 1-го и 2-го типа, третий клиент 3 сообщения любого типа, четвертый, сколько останется и удаляет очередь.

Количество отправленных и принятых сообщений должно совпадать. Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов.

Лабораторная работа «Вспомнить всё».

Использование разделяемой памяти и семафоров

Область разделяемой памяти позволяют множеству процессов отображать часть своих виртуальных адресов в общую область памяти. Благодаря этому любой процесс может записывать данные в разделяемую область памяти, и эти данные будут доступны для чтения/записи другим процессам.

Область разделяемой памяти находится в адресном пространстве ядра, поэтому все процессы, подключившиеся к данной области, имеют доступ к единственному экземпляру данных, причем все изменения «видны» всем процессам «моментально». Области разделяемой памяти входят в группу механизмов межпроцессного взаимодействия (IPC), но примитивом синхронизации не являются. Для предотвращения конфликтов при совместном доступе к области разделяемой памяти используются внешние средства синхронизации, как правило, семафоры. Процесс, подключившийся к области разделяемой памяти, получает указатель на неё и использует его, так как будто это адрес массива, полученного в результате динамического распределения памяти.

В адресном пространстве ядра имеется таблица, в которой отслеживаются все области разделяемой памяти, созданные в ОС. Одна строка этой таблицы соответствует одной области разделяемой памяти, кроме прочих, в таблице имеются следующие столбцы:

- Ключ, целое положительное число, присвоенное области разделяемой памяти процессом, который его создал. Другие процессы могут, указывая этот ключ, «открывать» область разделяемой памяти и получать дескриптор для доступа к ней.
- UID и GID пользователя от имени, которого запущен процесс, создавший область разделяемой памяти. Процесс эффективный UID, которого совпадает с UID пользователя – создателя области разделяемой памяти разделяемой

памяти, имеет право удалять область разделяемой памяти и изменять параметры управления ею.

- UID и GID назначенного владельца. Обычно эти идентификаторы совпадают с идентификаторами создателя области разделяемой памяти, но создатель может изменить их на идентификаторы другого пользователя.
- Права доступа к области разделяемой памяти для чтения/записи по категориям «владелец», «группа» и «прочие».
- Размер области в байтах.
- Время, когда какой-либо процесс в последний раз присоединялся к области разделяемой памяти.
- Время, когда какой-либо процесс в последний раз отсоединялся от области разделяемой памяти.
- Время, когда какой-либо процесс в последний раз изменял управляющие параметры области разделяемой памяти.

Для работы с областями разделяемой памяти определены следующие системные вызовы:

- `shmget()` создание/открытие области;
- `shmat()` подключение области к адресному пространству процесса;
- `shmdt()` изъятие области из адресного пространства процесса;
- `shmctl()` удаление области или получение/изменение управляющих параметров.

Область разделяемой памяти — это устойчивый объект, т.е. при завершении процесса создателя она остается в пространстве ядра, и другие процессы могут продолжать с ней работать. Для удаления области необходимо использовать в явном виде системный вызов `shmctl()`.

Прототип системного вызова `shmget()`:

```
int shmget(key_t key, int size, int flag);
```

Эта функция создает/открывает область разделяемой памяти, смысл параметров `key` и `flag` совпадает с аналогичными параметрами вызова `msgget()`.

Аргумент `size` задает размер в байтах, вновь создаваемой области разделяемой памяти.

При удачном завершении возвращается положительный целочисленный дескриптор, который необходим другим функциям семейства `shm`, при неудачном завершении `-1`, переменная `errno` содержит код ошибки.

Прототип системного вызова `shmat()`:

```
void* shmat(int shmId, void* addr, int flag);
```

Эта функция подключает область разделяемой памяти, указанную аргументом `shmId`, его вернул вызов `msgget()`. Если это вновь создаваемая область разделяемой памяти, то собственно выделение памяти происходит при вызове `shmat()`. Большинство приложений устанавливает значение аргумента `addr` равное нулю, в этом случае ядро возвращает, первое подходящее значение виртуального адреса. Не нулевое значение `addr` устанавливается при необходимости хранения в области разделяемой памяти ссылочных объектов (например, манипуляции со связанным списком). В таких случаях каждый из процессов вызывающих `shmat()` должен указывать одно и то же значение `addr`, а значение аргумента `flag` должно быть равно `SHM_RND` – признак округления значения `addr` до границы страницы памяти.

Аргумент `flag` может иметь значение `SHM_RDONLY`, которое означает, что вызывающий процесс подключается к области только на чтение, если этот флаг не установлен, то процесс получает права чтения/записи выданные при создании этой области.

При удачном завершении возвращается виртуальный адрес области отображения разделяемой памяти, при неудачном завершении `-1`, переменная `errno` содержит код ошибки.

Прототип системного вызова `shmdt()`:

```
int shmdt(void* addr);
```

Эта функция изымает область разделяемой памяти из адресного пространства процесса, её параметр `addr` – это адрес, полученный из системного вызова `shmat()`.

При удачном завершении возвращается 0, при неудачном завершении -1, переменная `errno` содержит код ошибки.

Прототип системного вызова `shmctl()`:

```
int shmctl(int shmId, int cmd, struct shmid_ds* buf);
```

С помощью этой функции можно запрашивать/изменять управляющие параметры области разделяемой памяти или удалять её. Первый параметр `shmId` – это дескриптор области, его вернул вызов `msgget()`. Второй параметр `cmd` – это модификатор действий функции, его возможные значения:

- `IPC_STAT`, тогда вызов будет копировать управляющие параметры области разделяемой памяти в объект по адресу `buf`;
- `IPC_SET`, тогда вызов будет изменять управляющие параметры области разделяемой памяти на значения находящиеся в объекте по адресу `buf`;
- `IPC_RMID`, тогда область разделяемой памяти будет удалена из системы аргумент `buf` не используется;
- `SHM_LOCK`, тогда область разделяемой памяти станет недоступна другим процессам, требуются права супервизора;
- `SHM_UNLOCK` тогда область разделяемой памяти разблокирована, требуются права супервизора.

Если к удаляемой области присоединены другие процессы, то операция удаления будет отложена до отсоединения последнего из них.

При удачном завершении возвращается 0, при неудачном завершении -1, переменная `errno` содержит код ошибки.

Примитив синхронизации – набор семафоров, это третий и последний механизм IPC. В адресном пространстве ядра имеется таблица, в которой отслеживаются все наборы семафоров, созданные в ОС. Одна строка этой

таблицы соответствует одному набору семафоров, кроме прочих, в таблице имеются следующие столбцы:

- Ключ, целое положительное число, присвоенное набору семафоров процессом, который его создал. Другие процессы могут, указывая этот ключ, «открывать» набора семафоров и получать дескриптор для доступа к нему.
- UID и GID пользователя от имени, которого запущен процесс, создавший набор семафоров. Процесс эффективный UID, которого совпадает с UID пользователя – создателя набора семафоров, имеет право удалять набор семафоров и изменять параметры управления им.
- UID и GID назначенного владельца. Обычно эти идентификаторы совпадают с идентификаторами создателя набора семафоров, но создатель может изменить их на идентификаторы другого пользователя.
- Права доступа к набору семафоров для чтения/записи по категориям «владелец», «группа» и «прочие».
- Количество семафоров в наборе.
- Время последнего изменения значения семафора.
- Время последнего изменения управляющих параметров набора.
- Указатель на массив семафоров.

Семафоры в наборе обозначаются индексами массива, в терминах языка Си. Первый семафор в наборе имеет индекс 0, второй индекс 1 и т.д., в каждом семафоре содержатся следующие данные:

- Значение семафора.
- Идентификатор процесса, который последним оперировал семафором.
- Число заблокированных процессов, ожидающих увеличения значения семафора.
- Число заблокированных процессов, ожидающих обращения значения семафора в ноль.

Для работы с наборами семафоров определены следующие системные вызовы:

- `semget()` создание/открытие набор семафоров;
- `semop()` операции с набором семафоров;
- `semctl()` удаление набора семафоров или получение/изменение управляющих параметров.

Содержательная часть семафора — это число типа `int`, соответственно даже с использованием набора с одним семафором можно реализовывать многозначную логику. Кроме того одним вызовом `semop()` можно опросить несколько семафоров, далее на одном и том же семафоре несколько процессов могут «спать» в ожидании его увеличения до положительных значений и ожидать его обращения в ноль. Все эти возможности избыточны, на практике рекомендуется использовать семафоры с двоичной логикой, за один вызов `semop()` опрашивать один семафор, не смешивать «ожидание нуля» и «ожидание плюса», управление состояние набора семафоров возложить на отдельный процесс. Любые усложнения алгоритм работы, например, возвращение значения семафора, только при обоснованной необходимости.

Набор семафоров — это устойчивый объект, т.е. при завершении процесса создателя он остается в пространстве ядра, и другие процессы могут продолжать с ним работать. При удалении набора семафора, все заблокированные на нем процессы активизируются и получают код ошибки.

Прототип системного вызова `semget()`:

```
int semget(key_t key, int num, int flag);
```

Эта функция создает/открывает набор семафоров, смысл параметров `key` и `flag` совпадает с аналогичными параметрами вызова `msgget()`. Аргумент `num` задает количество семафоров во вновь создаваемом наборе. При подключении к существующему набору семафоров аргумента `num` нельзя указывать большим, чем при создании.

При удачном завершении возвращается положительный целочисленный дескриптор, который необходим другим функция семейства `sem`, при неудачном завершении `-1`, переменная `errno` содержит код ошибки.

Прототип системного вызова `semop()`:

```
int semop(int semId, struct sembuf* opPtr, int len);
```

С помощью этой функции можно изменять значения одного/нескольких семафоров в наборе и/или проверять их на равенство нулю. Первый параметр `semId` – это дескриптор набора семафоров, его вернул вызов `semget()`. Вторым параметром `opPtr` – это указатель на массив объектов типа `struct sembuf`, каждый из которых задает одну операцию на изменение/запрос значения семафора. Последний параметр `len` показывает количество элементов в массиве, указанном параметром `opPtr`.

Тип данных `sembuf` определен следующим образом:

```
struct sembuf{
    short sem_num; //индекс семафора
    short sem_op;  //операция над семафором
    short sem_flg; //флаг операции
};
```

Если в поле `sem_op` содержится положительное/отрицательное число, то предпринимается попытка увеличения/уменьшения значения семафора на заданную величину, если в поле `sem_op` содержится 0, то выполняется проверка значения семафора на равенство нулю. Процесс блокируется при уменьшении значения семафора до отрицательных значений и в случае неравенства нулю при проверке. Вызов `semop` становится неблокирующим, если в поле `sem_flg` указан флаг `IPC_NOWAIT`, кроме того, поле может содержать значение `SEM_UNDO`.

При удачном завершении возвращается ноль, при неудачном завершении – 1, переменная `errno` содержит код ошибки.

Прототип системного вызова `semctl()`:

```
int semctl(int semId, int num, int cmd, union semun arg);
```

С помощью этой функции можно запрашивать и изменять управляющие параметры набора семафоров, удалять набор семафоров и производить ряд других действий. Первый параметр `semId` – это дескриптор набора семафоров, его вернул вызов `semget()`. Вторым параметром `num` – это индекс семафора, с

которым производятся манипуляции. Третий параметр `cmd`, задает тип операции, которая производится. Последний параметр `arg` – это объект типа `union`, который используется для задания управляющих параметров операции с одним/несколькими семафорами.

Тип данных `semun` определен следующим образом:

```
union semun{
    int val;                //значение семафора
    struct semid_ds *buf;   //управляющие параметры набора
    unsigned short *array; //массив значений семафоров
};
```

Поскольку `semun` это объединение, то его размер составляет 4 байта.

В таблице 3 перечислены значения параметра `cmd` и их смысл.

Таблица 3.

Варианты использования вызова `semctl()`

cmd	Описание операции
1	2
IPC_STAT	Скопировать управляющие параметры набора семафора в объект по адресу <code>arg.buf</code> . Аргумент <code>num</code> не используется.
IPC_SET	Заменить управляющие параметры набора семафора данными из объекта по адресу <code>arg.buf</code> . Аргумент <code>num</code> не используется.
IPC_RMID	Удалить набор семафоров. Аргументы <code>num</code> и <code>arg</code> не используется.
GETALL	Получить значения всех семафоров в массив по адресу <code>arg.array</code> . Аргумент <code>num</code> не используется.
SETALL	Установить значения всех семафоров из массива по адресу <code>arg.array</code> . Аргумент <code>num</code> не используется.
GETVAL	Вернуть значение семафора с номером <code>num</code> . Аргумент <code>arg</code> не используется.
SETVAL	Установить семафор с номером <code>num</code> равным параметру <code>arg.val</code> .

GETPID	Вернуть идентификатор процесса, который последним выполнял над семафором с номером num. Аргумент arg не используется.
--------	---

Продолжение таблицы 3.

1	2
GETNCNT	Вернуть количество процессов, которые заблокированы в ожидании увеличения значения семафора с номером num. Аргумент arg не используется.
GETZCNT	Вернуть количество процессов, которые заблокированы в ожидании обнуления семафора с номером num. Аргумент arg не используется.

При удачном завершении возвращается значение соответствующее заданному cmd, причем это может быть и отрицательное значение. При неудачном завершении -1, переменная errno содержит код ошибки.

В примере creator создаёт область разделяемой памяти, набор семафоров из одного семафора и выставляет его значение блокирующим (приравнивает 0). Процесс writer1 открывает область памяти и набор семафоров, затем спит на семафоре и при его открытии дважды записывает свою строку в область разделяемой памяти.

Процесс polisman открывает только набор семафоров, в бесконечном цикле ожидает ввода с клавиатуры. В зависимости от введенного значения:

- выводит количество заблокированных процессов;
- открывает семафор;
- удаляет набор семафоров.

Процесс reader1 подключается к области разделяемой памяти только на чтение. В бесконечном цикле копирует содержимое области памяти и выводит прочитанное на экран. Кроме того, он проверяет наличие набора семафоров, если набора нет, то отсоединяет область памяти, удаляет её и завершает свою работу.

```
// creator.cpp
int main()
{ int shmId, semId, keySem, keyShm;
  unsigned short int ray[1]={0};
  union semun{
    int val;
    unsigned short int *array;
  }ar;
  keySem = ftok("creator", 2);
  keyShm = ftok("creator", 1);
  printf("keySem=%d, keyShm=%d \n", keySem, keyShm);
  shmId = shmget(keyShm, 200, IPC_CREAT|0606);
  printf("shmId=%d\n", shmId);
  semId = semget(keySem, 1, IPC_CREAT|0606);
  printf("semId=%d\n", semId);
  ar.array = ray;
  semctl(semId, 0, SETALL, ar);
  exit(0);
}
```

```
// writer1.cpp
int main()
{ int j, shmId, semId, keySem, keyShm;
  char *addr1, buff[] = "    writer1 str";
  struct sembuf check_sem[1];
  keySem = ftok("creator", 2);
  keyShm = ftok("creator", 1);
  printf("keySem=%d, keyShm=%d \n", keySem, keyShm);
  check_sem[0].sem_num = 0;
  check_sem[0].sem_op = -1;
```

```

check_sem[0].sem_flg = 1;
semId = semget(keySem, 1, 0);
printf("semId=%d\n", semId);
shmId = shmget(keyShm, 200, 0);
printf("shmId=%d\n", shmId);
addr1 = (char*)shmat(shmId, 0, 0);
for(j = 0; j < 2; j++)
{
    semop(semId, check_sem, 1);
    printf("%s%d\n", buff, j);
    sprintf(addr1+(strlen(buff)+1)*j, "%s%d", buff, j);
}
shmdt(addr1);
exit(0);
}

```

```
// polisman.cpp
```

```

int main()
{
    int dr, i, semId, keySem, keyShm = 9;
    struct sembuf check_sem[1];
    keySem = ftok("creator", 2);
    printf("keySem=%d, keyShm=%d \n", keySem, keyShm);
    semId = semget(keySem, 1, 0);
    printf("semId=%d\n", semId);
    check_sem[0].sem_num = 0;
    check_sem[0].sem_op = 1;
    check_sem[0].sem_flg = 1;
    printf("нажмите -1 для завершения работы\n");
    printf("нажмите 9 для количества заблокированных\n");
    printf("нажмите 0 для открытия семафора\n");
}

```

```

for(;;)
{ scanf("%d", &dr);
  if(dr == 9)
  { i = semctl(semId, 0, GETNCNT, NULL);
    printf("на 0-м семафоре заблокировано %d \n", i);
  }
  if(dr == -1)
  { semctl(semId, 0, IPC_RMID, NULL);
    printf("завершение работы\n");
    exit(0);
  }
  if(dr == 0)
    semop(semId, check_sem, 1);
}
}

```

```
// reader1.cpp
```

```

int main()
{ int i = 9, shmId, semId, keySem = 8, keyShm;
  char *addr1 = NULL, buff[19];
  keySem = ftok("creator", 2);
  keyShm = ftok("creator", 1);
  printf("keySem=%d, keyShm=%d \n", keySem, keyShm);
  semId = semget(keySem, 1, 0);
  printf("semId=%d\n", semId);
  shmId = shmget(keyShm, 200, 0);
  printf("shmId=%d\n", shmId);
  addr1 = (char*)shmat(shmId, 0, SHM_RDONLY);
}

```

```

for(;;)
{ strcpy(buff, addr1);
  printf("\r%s ", buff);
  i = semctl(semId, 0, GETVAL, NULL);
  if(i == -1)
  { shmctl(addr1);
    shmctl(shmId, IPC_RMID, NULL);
    exit(0);
  }
}
}

```

В А (таблица 4) областей разделяемой памяти В процессов пишут произвольное количество своих строк. С каждой областью ассоциированы: семафор на запись и процесс–читатель с прямым доступом. Дополнительно есть С процессов–читателей с доступом через семафоры. Реализовать возможность одновременного открытия нескольких семафоров и дозапись в область памяти.

Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов.

Лабораторная работа «Аргументы, параметры».

Ассемблер из Си

Стандартным отладчиком в UNIX является утилита `gdb`, изначально разработанная под интерфейс командной строки. В среде рабочего стола KDE имеется графический интерфейс к отладчикам `gdb` и `xlsgdb` под названием `KDbg`. Это приложение в стандартных репозиториях отсутствует, способ установки на официальном сайте или по ссылке:

<https://itshaman.ru/articles/3613/8-luchshikh-otladchikov-linux-dlya-razrabotchikov-programmnogo-obespecheniya>

После установки `KDbg` рекомендуется выбрать синтаксис ассемблера в стиле Intel, Настройки/Общие настройки/Disassembly Flavor/Intel.

Соглашения о передаче параметров при вызове функций в UNIX подобных системах для 64 битных Intel совместимых процессоров подробно описано в документе «System V Application Binary Interface AMD64 Architecture Processor Supplement», объемом в 150 страниц. Это не общепринятый стандарт, не руководящий документ организации (в настоящее время удален с сайта фирмы Intel) и даже не статья в академическом смысле — это просто некий информационный ресурс.

Зафиксирован, следующий порядок передачи целочисленных аргументов: первые 6-ть аргументов помещаются в регистры `rdi`, `rsi`, `rdx`, `rcx`, `r8` и `r9`, следующие аргументы помещаются в стек в обратном порядке. Целочисленный результат (число или адрес) возвращаются через регистр `rax`. Зафиксирован, следующий порядок передачи аргументов с плавающей запятой (типы данных `float` или `double`): первые 8-м аргументов помещаются в регистры `xmm0`, `xmm1`, `xmm2`, `xmm3`, `xmm4`, `xmm5`, `xmm6` и `xmm7`, следующие аргументы помещаются в стек особым образом. Полученный результат (`float` или `double`) возвращаются через регистр `xmm0`. Аргументы типа `long double` помещаются в стек, результат

возвращается через регистр FPU. Кроме того, для всех вариантов необходимо гарантировать кратность 16-ти значения регистра стека. К сожалению, компиляторы не обязаны следовать описанным соглашениям, поэтому при необходимости использования ассемблера рекомендуется следующая последовательность действий.

На первом этапе создается минимально функциональный проект много файловой компиляции. В примере (файл callPr.c) вызываются функции, объявленные как внешние. Содержательная часть функций (файл addCalc.c) тривиальна складываются/делятся последний и первый аргументы, результаты сложения остальных аргументов игнорируются. Для компиляции используется makefile файл, который сначала преобразует addCalc.c в объектный файл, а потом линкует внешние функции с вызывающим модулем. Запуск вызывающего модуля выводит на экран требуемый результат.

```
//callPr.c
#include <stdio.h>
extern int addCalc(int,int,int,int,int,int,int,int,int);
extern double delCalc(double, double, double, double,
                     double, double, double, double, double);
int main()
{   int aCalc;
    double dCalc;
    aCalc = addCalc(1, 2, 3, 4, 5, 6, 7, 8, 9);
    printf("%d\n", aCalc);
    dCalc = delCalc(11.1, 22.2, 33.3, 44.4, 55.5, 66.6,
                   77.7, 88.8, 99.9);
    printf("%lf\n", dCalc);
}

//addCalc.c
int addCalc(int ar1, int ar2, int ar3, int ar4, int ar5,
            int ar6, int ar7, int ar8, int ar9)
```

```

{   int c19, c;
    c = ar2 + ar3 + ar4 + ar5 + ar6+ ar7 + ar8;
    c19 = ar9 + ar1;
    return(c19);
}

double delCalc(double ar1, double ar2, double ar3,
               double ar4, double ar5, double ar6, double ar7,
               double ar8, double ar9)
{   double c19, c;
    c = ar2 + ar3 + ar4 + ar5 + ar6 + ar7 + ar8;
    c19 = ar9 / ar1;
    return(c19);
}

//makefile
callPr: callPr.c addCalc.o
    gcc -g -o callPr callPr.c addCalc.o
addCalc.o: addCalc.c
    gcc -g -c addCalc.c

```

На втором этапе исходные файлы компилируются в ассемблерный текст, в весьма объемный текст, поэтому здесь не приводятся. Рекомендуется сравнить его с дизассемблерным текстом KDbg. Целью анализа полученных файлов является получение способа передачи аргументов в каждом конкретном случае.

```

//compilAsm
gcc -S -masm=intel addCalc.c
gcc -S -masm=intel callPr.c

```

На третьем этапе в ассемблерный файл с внешними функциями вносятся необходимые изменения, а затем модифицированный файл линкуется с

вызывающим модулем. Запуск вызывающего модуля должен выводить на экран корректный результат.

```
//addCalcMod03.s
    .intel_syntax noprefix
    .text
    .globl  addCalc, delCalc
    .type   addCalc, @function
addCalc:
    push    rbp
    mov     rbp, rsp
    mov     eax, [rbp + 32]
    add     eax, edi
    pop     rbp
    ret

    .type   delCalc, @function
delCalc:
    push    rbp
    mov     rbp, rsp
    movsd   xmm1, xmm0
    movsd   xmm0, [rbp + 16]
    divsd   xmm0, xmm1
    pop     rbp
    ret
//compilC
gcc -g -o callPr callPr.c addCalcMod03.s
```

Одна из внешних функций принимает $6 + A$ (таблица 4) целочисленных аргументов разных типов, включая адреса. Вторая функция принимает $8 + B$ дробных аргументов, включая и float и double. Вычислительную процедуру

оптимизировать в регистрах SSE. Проверить влияние ключей оптимизации на способ передачи аргументов и размеры исполняемых файлов, откомпилированных с отладочной информацией и без (ключ компиляции `-g`). Предъявить снимки экрана с дизассемблерным текстом внешних C функций.

Лабораторная работа «Поток сознания».

Многопоточное программирование

Одним из способов повышения производительности вычислительных систем является использование многопроцессорных платформ. Многопроцессорная система позволяет одновременно выполнять на каждом из процессоров отдельный процесс. Кроме того, появляется возможность распределить код одного процесса на несколько процессоров, сокращая общее время выполнения процесса. Такая техника получила название многопоточное (multi thread) программирование.

В многопоточном программировании особое внимание уделяется синхронизации работы нескольких потоков. Правильно спроектированное многопоточное приложение использует аппаратные ресурсы эффективнее. В частности, если многопоточное приложения запускается на системе с M процессорами, то все его потоки могут выполняться одновременно, каждый на отдельном процессоре. Следовательно, производительность такого приложения можно увеличить примерно в N раз, где N – число свободных в данный момент процессоров ($N \leq M$).

Производительность многопоточного приложения можно повысить даже в том случае, если оно запускается на однопроцессорной системе. Например, если один из потоков приложения блокируется каким-то системным вызовом, на этом процессоре может выполняться другой поток. Таким образом, сокращается общее время выполнения приложения.

Необходимо отметить характерные черты потоков:

- поток - есть принадлежность процесса;
- все потоки разделяют адресное пространство процесса;
- изменения глобальной переменной процесса видимы во всех потоках.

Функции управления потоками описаны в стандартах POSIX.1b и POSIX.1c. Поток состоит из следующих элементов:

- идентификатора потока;
- динамического стека;
- набора регистров (счетчик команд, указатель стека);
- сигнальной маски;
- значения приоритета;
- специальной памяти.

Для работы с потоками можно использовать следующие системные вызовы:

- `pthread_create()` создание потока;
- `pthread_exit()` завершение потока;
- `pthread_kill()` отправка сигнала потоку управления;
- `sched_yield()` передача управления другому потоку;
- `pthread_join()` ожидание завершения потока;
- `pthread_detach()` отсоединение потока;
- `pthread_self()` получение идентификатора потока;
- `pthread_attr_init()` инициализация атрибутов потока.

Поток выполнения создается функцией `pthread_create()`. Каждому потоку присваивается идентификатор, уникальный среди всех потоков процесса. Вновь созданный поток наследует сигнальную маску процесса, динамический стек, значение приоритета и набор регистров. Динамический стек и регистры (счетчик команд и указатель стека) позволяют потоку выполняться независимо от других потоков. Поток может изменить унаследованную сигнальную маску и выделить динамическую память для хранения своих данных.

Потоку при создании назначается соответствующая функция. Поток завершается, когда эта назначенная функция возвращает результат или когда поток вызывает функцию `pthread_exit()`. Когда в процессе создается первый поток, то фактически создаются два потока: один – для выполнения указанной функции, а другой - для продолжения выполнения процесса. Процесс

завершается, когда функция `main()` возвращает результат или при завершении последнего потока.

Все потоки в процессе совместно используют одни сегменты данных и кода. Когда один поток записывает данные в глобальные переменные в процессе, остальные потоки сразу же видят эти изменения. Если какой-либо поток вызывает функцию `exit()` или `_exit()`, то завершаются все потоки и сам процесс. Поэтому завершающийся поток, если он не хочет разрушить процесс, в котором выполняется, при завершении должен вызывать функцию `pthread_exit()`. Функция `main()` многопоточного приложения тоже должна завершаться вызовом `pthread_exit()`.

Функция `pthread_kill()` посылает сигнал указанному потоку управления. Номер сигнала указывается вторым аргументом, если этот аргумент равен нулю, то выполняются проверки на ошибку, но сигнал фактически не посылается. В большинстве случаев поток реагирует на пришедший сигнал завершением работы, но можно и изменить реакцию на сигнал.

Потоку присваивается целочисленное значение приоритета, чем больше это значение, тем чаще планируемое выполнение потока. Поток может намеренно передать выполнение другим потокам с таким же приоритетом; для этого используется функция `sched_yield()`. Кроме того, поток может ожидать завершения другого потока и получить его код возврата с помощью функции `pthread_join()`.

По умолчанию потоки создаются «способными к присоединению», хорошей практикой считается явное описание статуса потока как присоединенного (`joinable`) или отсоединенного (`detached`). Это делается с помощью системных вызовов `pthread_join()` и `pthread_detach()`. После объявления статуса потока `joinable/detached` повторно его изменить нельзя.

Различия в статусах заключаются в управлении ресурсами по завершению потока. Присоединяемый поток после своего завершения не освобождает память до вызова `pthread_join()`, который освобождает всю занимаемую память (стек и

т.п.) и возвращает результат выполнения потока. Отсоединенный поток этого не требует – все ресурсы возвращаются системе непосредственно по его завершению, узнать результат в таком случае невозможно ввиду того, что нет переменной, в которую этот результат может быть сохранен.

Системный вызов `pthread_attr_init()` инициализирует атрибуты вновь создаваемого потока значениями по умолчанию.

Прототип системного вызова `pthread_create()`:

```
int pthread_create(pthread_t* thr, pthread_attr_t* attr,
                  void* (*start_routine)(void*), void* arg);
```

Эта функция создаёт новый поток для выполнения функции, адрес которой задан аргументом `start_routine`. Функция, указанная в этом аргументе, должна принимать один входной параметр типа `void*` и возвращать данные такого же типа. Фактический аргумент, который передается в `start_routine` функцию в начале выполнения нового потока, указывается в аргументе `arg`.

Идентификатор нового потока возвращается через аргумент `thread`. Если этому аргументу присвоено значение `NULL`, идентификатор не возвращается. Аргумент `attr` содержит атрибуты, присваиваемые вновь создаваемому потоку. Значение этого аргумента может быть равно `NULL`, если новый поток должен использовать атрибуты по умолчанию, или адресу объекта содержащего атрибуты. При удачном завершении функции возвращается 0, при неудачном код ошибки.

Прототип системного вызова `pthread_attr_init()`:

```
int pthread_attr_init(pthread_attr_t *attr);
```

Функция `pthread_attr_init()` записывает в описатель атрибутов потока исполнения, на который указывать аргумент `attr`, значения атрибутов принятые в системе по умолчанию. Далее проинициализированный описатель атрибутов может изменяться и использоваться для создания потока исполнения. При удачном завершении функции возвращается 0, при неудачном код ошибки.

Прототип системного вызова `pthread_self()`:


```
pthread_t pthread_self(void);
```

Возвращает идентификатор потока самому потоку. Всегда завершается успешно.

Прототип системного вызова pthread_exit():

```
void pthread_exit(void *retval);
```

Завершает поток. Значение аргумента retval – адрес статической переменной, которая содержит код возврата завершаемого потока. Если никакой другой поток не будет использовать код завершающегося потока, значение переменной retval можно указать как NULL. Всегда завершается успешно.

Прототип системного вызова pthread_kill():

```
int pthread_kill(pthread_t thread, int signo);
```

Посылает сигнал потоку, номер сигнала содержится в signo. При успешном завершении функция возвращает 0, при неудачном код ошибки.

Прототип системного вызова pthread_join():

```
int pthread_join(pthread_t thread, void** thr_return)
```

Вызывается для ожидания завершения потока. В переменной thr_return возвращает код возврата потока, переданный через вызов функции pthread_exit(). При удачном завершении функции возвращается 0, при неудачном код ошибки.

Прототип системного вызова int pthread_detach(pthread_t thread):

```
int pthread_detach(pthread_t thread)
```

Присваивает потоку статус отсоединенного потока.

Прототип системного вызова sched_yield()

```
int sched_yield(void);
```

Поток вызывает функцию sched_yield, чтобы передать выполнение другим потокам с таким же приоритетом. При успешном выполнении данная функция возвращает 0, а в случае неудачи –1, переменная errno содержит код ошибки.

Если операционная система выполняется на машине, где установлено более одного процессора, то по умолчанию, поток выполняется на любом доступном процессоре. Однако в некоторых случаях, набор процессоров, на

первый параметр — это указатель на экземпляр структуры атрибутов потока, второй – размер переменной `cpuset`, третий – указатель на экземпляр `cpuset`.

Системный вызов `pthread_attr_setaffinity_np()` позволяет изменить атрибуты вновь создаваемого потока.

```
int pthread_attr_setaffinity_np(pthread_attr_t *at,
                                size_t size, const cpu_set_t *cpuset);
```

первый параметр — это указатель на экземпляр структуры атрибутов потока, второй – размер переменной `cpuset`, третий – указатель на экземпляр `cpuset`.

Все четыре вызова при удачном завершении возвращают 0, при неудачном завершении -1, переменная `errno` содержит код ошибки.

В примере создаётся переменная с атрибутами потока, назначается процессор – исполнитель, запускается поток вычисления, а `main` – поток ожидает его завершения.

```
#define _GNU_SOURCE
const int CALC_COUNT = 200000000;
//-----
void* thread_func1(void*)
{ long int i, j = 0;
  double rez;

  for (i=0; i < CALC_COUNT; i++, j++)
    rez=i*3+sin(i)-sqrt(1/15);
  printf("\nпоток 1 завершился");
  pthread_exit(NULL);
}
//-----
int main(void)
{ int result;
  pthread_t id1;
  pthread_t thread1, thread2, thread3;
```

```

cpu_set_t set;
pthread_attr_t attr;

pthread_attr_init(&attr);
CPU_ZERO(&set);
CPU_SET(0, &set);
result = pthread_attr_setaffinity_np(&attr,
                                     sizeof(cpu_set_t), &set);
result=pthread_create(&thread1,&attr,thread_func1,
                    NULL);

result = pthread_join(thread1, NULL);
idl = pthread_self();
printf("\nmain поток %lu завершился\n", idl);
pthread_exit(NULL);
}

```

На компьютере с N процессорами замерить вручную время работы программы с 1, 2, ...N+1 потоками исполнения с привязкой к процессорам и без неё. Загруженность процессоров оценить в системном мониторе. Результаты свести в таблицу и/или график. Замеры времени произвести дважды, второй раз использовать внешнюю функцию, оптимизированную в регистрах SSE.

Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов.

Лабораторная работа «Мутанты среди нас».

Использование мьютексов

Объекты синхронизации потоков выполнения.

Потоки выполнения в процессе совместно используют то же адресное пространство, что и процесс. Это значит, что все глобальные и статические переменные процесса доступны для всех его потоков. Чтобы обеспечить безошибочное манипулирование этими переменными, потоки должны синхронизировать свою работу. В частности, ни один поток не должен обращаться к переменной, значение которой в текущий момент изменяется другим потоком, и ни один поток не может изменять значение переменной в то время, когда его читают другие потоки.

Синхронизация нужна и тогда, когда два и более потока выполняют операции с общим потоком ввода-вывода. Например, если два потока выполнения одновременно записывают информацию в поток вывода, то невозможно предсказать, какие именно данные попадут в этот поток.

Процессы, потоки которых применяют какие-либо примитивы синхронизации, должны определить для них область памяти в своем виртуальном адресном пространстве. Соответственно при завершении процесса эти объекты будут уничтожены системой, однако хорошей практикой считается их явное удаление.

Взаимоисключающие блокировки (mutex – сокращение от MUTual EXclusion) позволяют организовать выполнение потоков так, что, когда несколько потоков пытаются установить такую блокировку, успеха достигает только один из них, который и продолжает выполняться. Остальные потоки блокируются до тех пор, пока эта блокировка не будет снята потоком владельцем. При снятии взаимоисключающие блокировки невозможно предсказать, какой ожидающий поток будет следующим владельцем блокировки.

Для работы с мьютексами определены следующие системные вызовы:

- `pthread_mutex_init();`
- `pthread_mutex_lock();`
- `pthread_mutex_trylock();`
- `pthread_mutex_unlock();`
- `pthread_mutex_destroy();`

Прототип системного вызова `pthread_mutex_init()`:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const
                        pthread_mutexattr_t *mutexattr);
```

Инициализация взаимоисключающей блокировки. Переменная `mutex` типа `pthread_mutex_t` определяется вызывающим потоком. Аргумент `mutexattr` типа `pthread_mutexattr_t` – это указатель на объект, содержащий атрибуты для новой блокировки, если эта блокировка должна иметь атрибуты по умолчанию, то значение аргумента `mutexattr` равно нулю. Вместо функции `pthread_mutex_init` можно использовать статические инициализаторы:

```
static pthread_mutex_t fastmutex =
                                PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t recmutex =
                                PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP;
static pthread_mutex_t errchkmutex =
                                PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP;
```

Указанные макросы `fast` (быстрый), `recursive` (рекурсивный) и `error check` (контроль ошибок) изменяют поведение мьютексовых функций в случае `deadlock` – тупиковых ситуаций. В справочной системе (`man` – страницах) имеется подробное описание макросов `recursive` и `error check`. В большинстве случаев используются «быстрые» мьютексы, их поведение рассматривается далее.

Прототип системного вызова `pthread_mutex_lock()`:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

Захват взаимноисключающей блокировки. Если мьютекс уже захвачен, то поток блокируется до его освобождения.

Прототип системного вызова `pthread_mutex_trylock()`:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

Не блокирующий захват взаимноисключающей блокировки. Если мьютекс уже захвачен, то функция `pthread_mutex_trylock` возвращает код ошибки `EBUSY`, не блокируя поток.

Прототип системного вызова `pthread_mutex_unlock()`:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Освобождение взаимноисключающей блокировки.

Прототип системного вызова `pthread_mutex_destroy()`:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Удаление объекта – взаимноисключающая блокировка.

Все перечисленные функции возвращают 0 в качестве успешного завершения, либо не нулевой код ошибки.

При использовании взаимноисключающих блокировок следует избегать создания тупиковых ситуаций. Такая ситуация может возникнуть, когда процесс использует несколько взаимноисключающих блокировок и два или более потоков в этом процессе пытаются стать владельцами этих блокировок в произвольной последовательности. Ниже приведен псевдокод тупиковой ситуации и его описание:

поток X	поток Y
...	...
lock(A)	lock(B)
lock(B)	lock(A)
...	...
unlock(B)	unlock(A)
unlock(A)	unlock(B)
...	...

В процессе инициализированы две взаимоисключающие блокировки (А и В), которыми пользуются два потока (Х и Y). Допустим, поток Х установил блокировку А, а поток Y - блокировку В. Когда Х пытается стать владельцем блокировки В, он блокируется, потому что эта блокировка установлена потоком Y. Если затем поток Y пытается стать владельцем блокировки А, он тоже блокируется. Таким образом, создается тупиковая ситуация: блокируются оба потока, потому что каждый из них пытается установить блокировку, принадлежащую другому потоку. Ни один из этих потоков не будет выполняться, потому что владелец блокировки заблокирован и не может снять ее.

На первый взгляд использование функции `pthread_mutex_trylock()` вместо `pthread_mutex_lock()` исключает тупиковую ситуацию, но это не так. На самом деле простая замена `pthread_mutex_lock()` на `pthread_mutex_trylock()`, приводит к тому, что логика обоих потоков выполняться все равно не будет, хотя потоки и не заблокированы. Это ситуация даже хуже, чем deadlock, т.к. потоки впустую растрачивают ресурсы процессора. Общая рекомендация в подобных случаях – избегать вложения взаимоисключающих блокировок друг в друга.

В примере поток `main` инициализирует мьютекс, создаёт два потока, ожидает завершения одного из них, деактивирует мьютекс и завершает свою работу. Потоки записывают в глобальную переменную, каждый свою строку и выводят содержание глобальной переменной на консоль. Процедура изменения глобальной переменной защищена мьютексом.

```
char buff[100];
pthread_mutex_t mtx;
//-----
void strCpySleep(char *dst, char* src)
{
    int i, j;
    double rez = 0;
    for(j = 0; j < 10; j++)
```



```

{
    for (i=0; i < 200000; i++)    //сравнить с
        rez=i*3+sin(i)-sqrt(1/15); //sched_yield()
    dst[j] = src[j];    }
    rez += 1.;}
//-----
void *thr_func1(void*)
{
    char tt[] = "ONE-THREAD";
    int i;
    for(i = 0; i < COUNT + 1; i++)
    {
        pthread_mutex_lock(&mtx);
        strCpySleep(buff, tt);
        printf("THREAD1 %s\n", buff); //обратить
        pthread_mutex_unlock(&mtx);    //внимание    }
        printf("THREAD 1 EXIT\n");
        pthread_exit(NULL);}
//-----
void *thr_func2(void*)
{
    char tt[] = "thread__two";
    int i;
    for(i = 0; i < COUNT; i++)
    {
        pthread_mutex_lock(&mtx);
        strCpySleep(buff, tt);
        pthread_mutex_unlock(&mtx);
        printf("\t\t\t\t2_thread %s\n", buff);    }
        printf("\t\t\t\t2_thread quit\n");
        pthread_exit(NULL);}
//-----

```

```

int main(void)
{
    int result;
    pthread_t id, thr1, thr2;
    result = pthread_mutex_init(&mtx, NULL);
    result = pthread_create(&thr1, NULL, thr_func1, NULL);
    result = pthread_create(&thr2, NULL, thr_func2, NULL);
    // result += pthread_join(thr1, NULL);    //поиграться с
    result += pthread_join(thr2, NULL);      //ожиданием
    pthread_mutex_destroy(&mtx);
    id = pthread_self();
    printf("main поток %lu завершился\n", id);
    pthread_exit(NULL);
}

```

Вывод на экран демонстрирует корректную работу потоков с глобальной переменной. Поток main поток не ждет завершения потока 1.

```

2_thread tread__two
THREAD1 ONE-THREAD
2_thread tread__two
2_thread tread__two
2_thread tread__two
2_thread tread__two
2_thread quit
main поток 3075712768 завершился
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD

```

```
THREAD1 ONE-THREAD
```

```
THREAD 1 EXIT
```

В одну глобальную переменную – строку А (таблица 4) потоков пишут и выводят каждый свою строку. Четные потоки пишут по В строк, нечетные по С строк. Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов.

Лабораторная работа «По радио объявили».

Использование условных переменных

Мьютексы используются для защиты критического участка кода, а условные переменные для ожидания некоторого события. Условную переменную можно понимать, как «вложенный мьютекс».

Работу условных переменных можно пояснить следующим образом. Сначала поток устанавливает взаимоисключающую блокировку, затем засыпает на условной переменной до момента выполнения определенного условия. Пока поток спит на условной переменной захваченный им мьютекс освобождается. Когда другой поток сигнализирует о том, что событие случилось, первый поток просыпается, снова блокирует мьютекс и выполняет проверку некоторого условия. Если оно не устраивает, поток опять блокируется на условной переменной. Если же условие выполняется, поток снимает взаимоисключающую блокировку и выполняется дальше.

Для работы с условными переменными определены следующие системные вызовы:

- `pthread_cond_init();`
- `pthread_cond_wait();`
- `pthread_cond_timedwait();`
- `pthread_cond_signal();`
- `pthread_cond_broadcast();`
- `pthread_cond_destroy();`

Системный вызов `pthread_cond_init()` позволяет инициализировать условную переменную, его прототип выглядит следующим образом:

```
int pthread_cond_init(pthread_cond_t *cond,  
                      pthread_condattr_t *cond_attr);
```

первый параметр — это указатель на экземпляр условной переменной, второй — указатель на экземпляр атрибутов, в большинстве случаев равен `NULL`.

Системный вызов `pthread_cond_wait()` позволяет блокировать поток на условной переменной, его прототип выглядит следующим образом:

```
int pthread_cond_wait(pthread_cond_t *cond,
                      pthread_mutex_t *mutex);
```

первый параметр — это указатель на инициализированную условную переменную, второй — указатель на инициализированный мьютекс.

Системный вызов `pthread_cond_timedwait()` позволяет блокировать поток на условной переменной на определенное время, его прототип выглядит следующим образом:

```
int pthread_cond_timedwait(pthread_cond_t *cond,
                             pthread_mutex_t *mutex, const struct timespec *abstime);
```

последний параметр — время блокировки.

Системный вызов `pthread_cond_signal()` позволяет послать одиночный сигнал разблокировки, его прототип выглядит следующим образом:

```
int pthread_cond_signal(pthread_cond_t *cond);
```

в результате проснется, какой-либо один поток, если событие его не устроит, он снова блокируется, а сигнал будет проигнорирован.

Системный вызов `pthread_cond_broadcast()` разблокирует все потоки, ожидающие условную переменную, его прототип выглядит следующим образом:

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

в результате просыпаются все потоки, потоки для которых, событие не удовлетворительно снова будут заблокированы.

Системный вызов `pthread_cond_destroy()` позволяет удалять условную переменную, его прототип выглядит следующим образом:

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

Все перечисленные функции возвращают 0 в качестве успешного завершения, либо не нулевой код ошибки.

В примере поток `main` инициализирует 2 мьютекса, 1 условную переменную, создает 2 потока и объявляет их отсоединенными. Затем он засыпает на комбинации мьютекс/условная переменная `mtx/cnd`. Потоки

записывают в глобальную переменную каждый свою строку, далее используя комбинацию мьютекс/условная переменная mtxCnd/cnd, они увеличивают значение глобального счетчика. Они завершают свою работу, когда значение глобального счетчика станет равно 2. Вместе с ними проснется поток main, он деактивирует мьютексы и условную переменную.

```
#define COUNT 5
#define SYNC_MAX_COUNT 2
char buff[100];
int sync_count = 0;
pthread_mutex_t mtx;
pthread_mutex_t mtxCnd;
pthread_cond_t cnd;
//-----

void strCpySleep(char *dst, char* src)
{
    int i, j;
    double rez = 0;
    for(j = 0; j < 10; j++)
    {
        for (i=0; i < 200000; i++)
            rez=i*3+sin(i)-sqrt(1/15);
        dst[j] = src[j];
    }
    rez += 1.;
}
//-----

void *thr_func1(void*)
{
    char tt[] = "ONE-THREAD";
    int i;
    for(i = 0; i < COUNT + 1; i++)
    {
```

```

        pthread_mutex_lock(&mtx);
        strCpySleep(buff, tt);
        printf("THREAD1 %s\n", buff);
        pthread_mutex_unlock(&mtx);
    }
    pthread_mutex_lock(&mtxCnd);
    sync_count++;
    if(sync_count < SYNC_NAX_COUNT)
        pthread_cond_wait(&cnd, &mtxCnd);
    else
        pthread_cond_broadcast(&cnd);
//        pthread_cond_signal(&cnd);
    pthread_mutex_unlock(&mtxCnd);
    printf("THREAD1 EXIT\n");
    pthread_exit(NULL);
}
//-----
void *thr_func2(void*)
{
    char tt[] = "thread__two";
    int i;
    for(i = 0; i < COUNT; i++)
    {
        pthread_mutex_lock(&mtx);
        strCpySleep(buff, tt);
        pthread_mutex_unlock(&mtx);
        printf("\t\t\t\t2_thread %s\n", buff);
    }
    pthread_mutex_lock(&mtxCnd);
    sync_count++;

```

```

    if(sync_count < SYNC_NAX_COUNT)
        pthread_cond_wait(&cnd, &mtxCnd);
    else
        pthread_cond_broadcast(&cnd);
    pthread_mutex_unlock(&mtxCnd);
    printf("\t\t\t\t2_thread quit\n");
    pthread_exit(NULL);
}
//-----
int main(void)
{ int result;
  pthread_t id, thr1, thr2;
  result = pthread_mutex_init(&mtx, NULL);
  result = pthread_mutex_init(&mtxCnd, NULL);
  result = pthread_cond_init(&cnd, NULL);
  result = pthread_create(&thr1, NULL, thr_func1, NULL);
  result = pthread_create(&thr2, NULL, thr_func2, NULL);
  pthread_detach(thr1);
  pthread_detach(thr2);
  pthread_mutex_lock(&mtx);
  pthread_cond_wait(&cnd, &mtxCnd);
  pthread_mutex_unlock(&mtx);
  pthread_cond_destroy(&cnd);
  pthread_mutex_destroy(&mtx);
  pthread_mutex_destroy(&mtxCnd);
  id = pthread_self();
  printf("main поток %lu завершился\n", id);
  pthread_exit(NULL);
}

```

Вывод на экран демонстрирует ожидание на условной переменной.


```

2_thread tread__two
THREAD1 ONE-THREAD
2_thread tread__two
2_thread tread__two
2_thread tread__two
2_thread tread__two
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 EXIT
main поток 3075778304 завершился
2_thread quit

```

В одну глобальную переменную – строку В (таблица 4) потоков пишут и выводят по С строк. Все потоки спят на условной переменной, которую изменяет main поток, в зависимости от консольного ввода. Условная переменная проверяет одно условие для четных потоков и другое для нечетных. Необходимо реализовать обработку ошибок для всех системных вызовов и вывести код ошибки для одного из вызовов.

Лабораторная работа «Всё серьёзно».

Не стандартные приемы работы с поточными примитивами синхронизации

В примере для демонстрации тупика используется системный вызов `sched_yield()`. Замена `pthread_mutex_lock()` на неблокирующий вызов `pthread_mutex_trylock()`, приводит к тому что потоки и логику свою не выполняют и не спят, в этом можно убедиться в помощью системного монитора. Поток `main()`, в зависимости от консольного ввода освобождает один из мьютексов и потоки больше не засыпают. Из листинга видно, что программа работает не корректно.

```
char buff[100];

pthread_mutex_tmtxA;
pthread_mutex_tmtxB;

//-----

void *thr_func1(void*)
{ char tt[] = "ONE-THREAD";
  int i;
  pthread_t id;
  id = pthread_self();
  printf("ONE-THREAD %lu\n", id);
  for(i = 0; i < 6; i++)
  { pthread_mutex_lock(&mtxB);
    sched_yield();
    for(;pthread_mutex_trylock(&mtxA) != 0;);
    strcpy(buff, tt);
    pthread_mutex_unlock(&mtxA);
```

```

pthread_mutex_unlock(&mtxB);
printf("THREAD1 %s\n", buff);
    }
printf("THREAD 1 EXIT\n");
pthread_exit(NULL);
}
//-----
void *thr_func2(void*)
{ char tt[] = "thread_two";
  int i;
  pthread_t id;
  id = pthread_self();
  printf("\t\t\t\t2_thread %lu\n", id);
  for(i = 0; i < 5; i++)
  { pthread_mutex_lock(&mtxA);
    sched_yield();
    for(;pthread_mutex_trylock(&mtxB) != 0;);
    strcpy(buff, tt);
    pthread_mutex_unlock(&mtxB);
    pthread_mutex_unlock(&mtxA);
    printf("\t\t\t\t2_thread %s\n", buff);
  }
  printf("\t\t\t\t2_thread quit\n");
  pthread_exit(NULL);
}

```

```
int main(void)
{
    int result, i, dr;
    pthread_t id, thr1, thr2;

    result = pthread_mutex_init(&mtxA, NULL);
    result = pthread_mutex_init(&mtxB, NULL);
    result = pthread_create(&thr1, NULL, thr_func1, NULL);
    result = pthread_create(&thr2, NULL, thr_func2, NULL);

    scanf("%d", &dr);
    if(dr == 9)
        pthread_mutex_unlock(&mtxA);
    if(dr == 1)
        pthread_mutex_unlock(&mtxB);

    result += pthread_join(thr1, NULL);
    result += pthread_join(thr2, NULL);

    pthread_mutex_destroy(&mtxA);
    pthread_mutex_destroy(&mtxB);
    id = pthread_self();
    printf("main поток %lu завершен\n", id);
    pthread_exit(NULL);
}
```

```

ONE-THREAD 3075836736

                2_thread 3067444032

9

THREAD1 ONE-THREAD

                2_thread thread_two
                2_thread thread_two
                2_thread thread_two
                2_thread thread_two
                2_thread thread_two

THREAD1 thread_two
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD
THREAD1 ONE-THREAD

                2_thread quit

THREAD 1 EXIT

main поток 3075839744 завершился

```

В одну глобальную переменную – строку С (таблица 4) потоков в бесконечном цикле пишут и выводят свои строки. Один поток работает с мьютексом через trylock. Ещё один поток спит на условной переменной некоторое время, а проснувшись не освобождает мьютекс. Поток main, зависимости от консольного ввода освобождает или разрушает мьютекс. Необходимо реализовать обработку ошибок для всех системных вызовов и вывести значение кода errno для одного из вызовов.

Приложение 1

Таблица 4

Варианты заданий

№	A	B	C
1	7	5	7
2	8	1	1
3	5	3	1
4	6	3	5
5	4	1	3
6	6	3	7
7	5	6	7
8	5	7	7
9	7	4	6
10	6	4	4
11	6	8	1
12	6	3	7
13	4	7	1
14	5	3	6
15	4	8	4
16	8	4	4
17	6	4	7
18	4	1	6
19	5	8	3
20	7	3	6
21	6	2	3
22	8	7	2
23	6	1	6
24	7	2	2
25	6	2	6
26	6	5	2
27	4	7	8
28	7	2	1
29	8	8	7
30	7	2	8
31	7	7	1
32	5	1	8
33	4	4	8

№	A	B	C
34	7	7	6
35	7	2	8
36	7	5	1
37	4	6	1
38	7	6	6
39	6	7	4
40	6	2	8
41	5	6	1
42	6	1	1
43	5	4	2
44	5	2	6
45	7	8	4
46	7	3	3
47	4	1	8
48	7	8	6
49	6	1	5
50	7	4	5
51	6	5	3
52	5	6	7
53	5	7	3
54	8	8	4
55	5	8	6
56	4	7	8
57	8	1	4
58	7	3	8
59	8	4	8
60	7	4	2
61	6	1	3
62	6	7	3
63	8	5	6
64	5	2	3
65	6	5	1
66	5	4	5

67	5	8	2
68	3	3	5
69	6	8	5
70	8	4	3
71	8	7	2
72	4	5	5
73	3	1	4
74	7	7	6
75	3	8	8
76	7	1	2
77	6	8	4
78	8	2	4
79	7	1	5
80	3	4	8
81	2	8	4
82	4	5	6
83	7	4	4
84	4	6	5
85	6	1	3
86	7	8	7
87	3	1	1
88	1	4	8
89	8	4	4
90	4	6	4
91	3	8	6
92	7	3	5
93	6	3	1
94	7	2	6
95	5	5	6
96	2	5	6
97	8	1	1
98	5	4	1
99	5	3	5

100	7	3	7
101	5	4	8
102	4	7	2
103	6	6	6
104	8	5	7
105	2	4	2
106	7	1	2
107	2	6	1
108	8	5	1
109	5	5	3
110	2	2	2
111	1	8	8
112	5	7	7
113	8	8	6
114	3	8	2
115	3	1	4
116	6	1	6
117	7	4	1
118	2	8	3
119	6	8	3
120	7	2	6
121	1	1	2
122	5	4	4
123	4	5	7
124	3	2	1
125	5	8	1
126	4	6	2
127	2	3	6
128	1	2	7
129	2	5	8
130	4	7	7
131	2	7	1
132	7	7	6

Приложение 2

Варианты схем

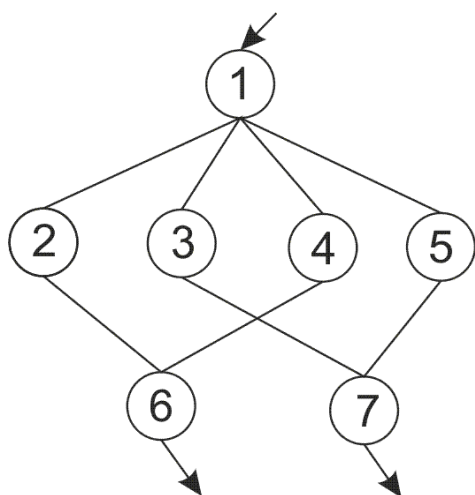


Схема № 1

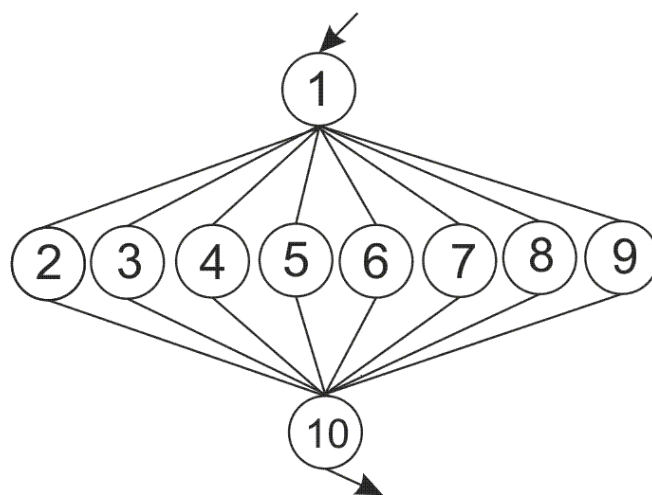


Схема № 2

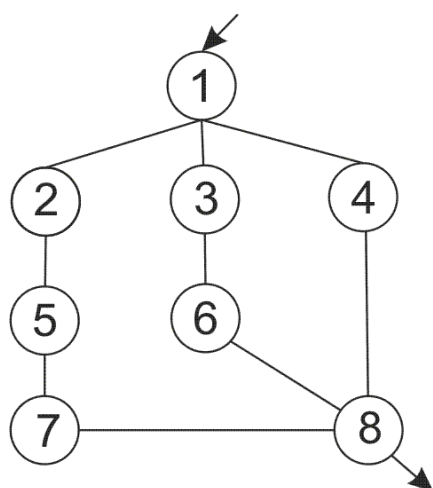


Схема № 3

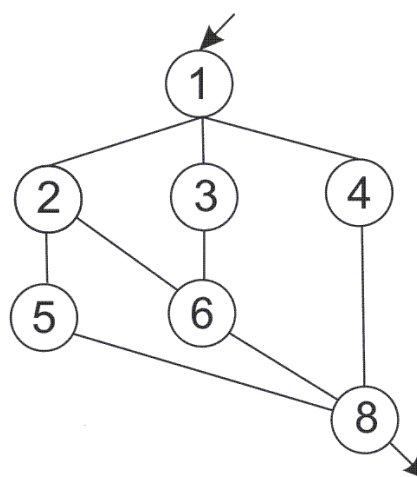


Схема № 4

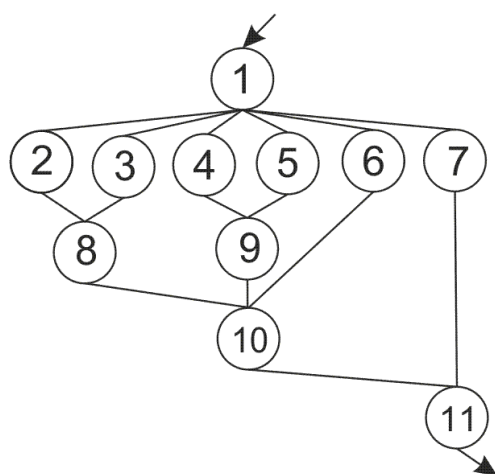


Схема № 5

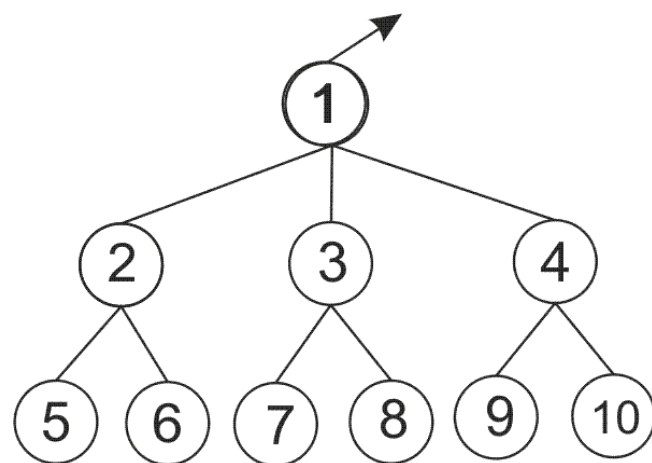


Схема № 6

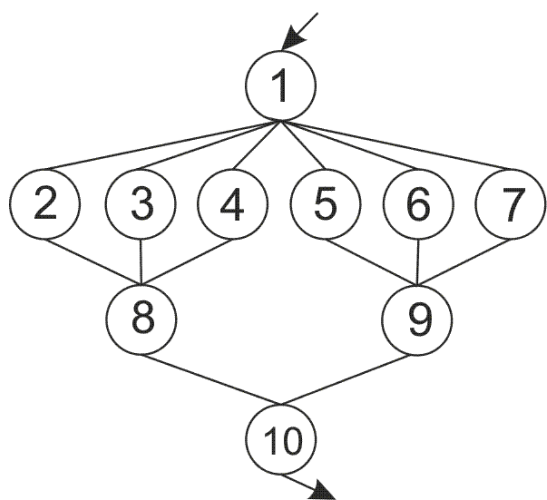


Схема №7

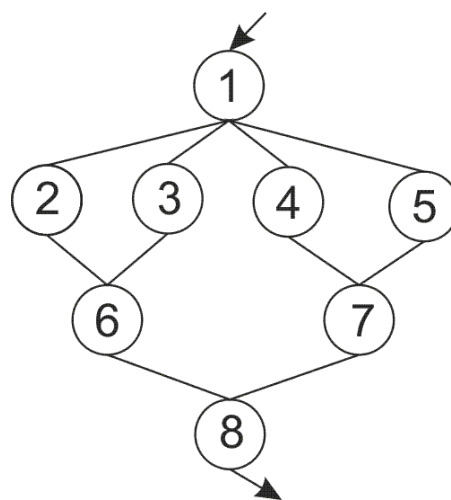


Схема №8

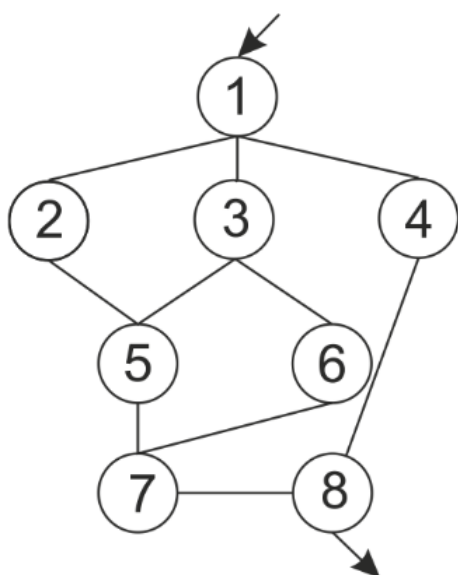


Схема № 9

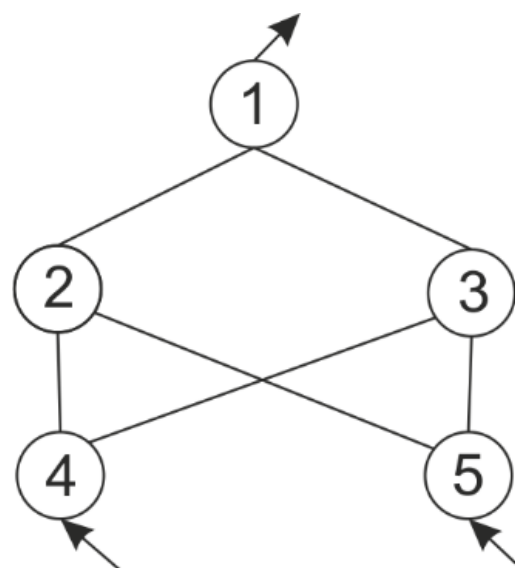


Схема № 10

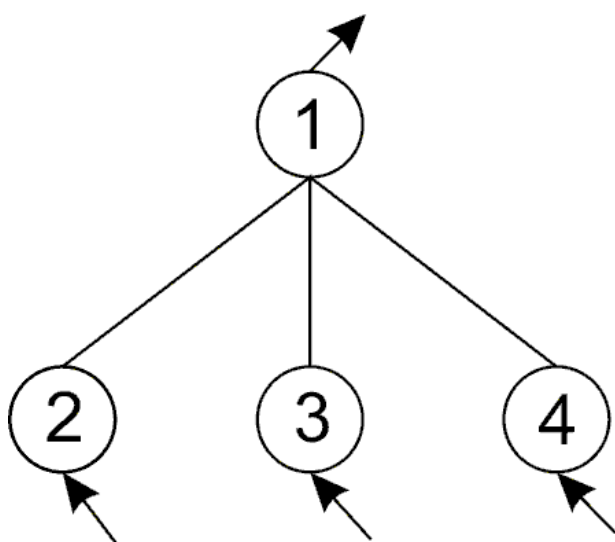


Схема № 11

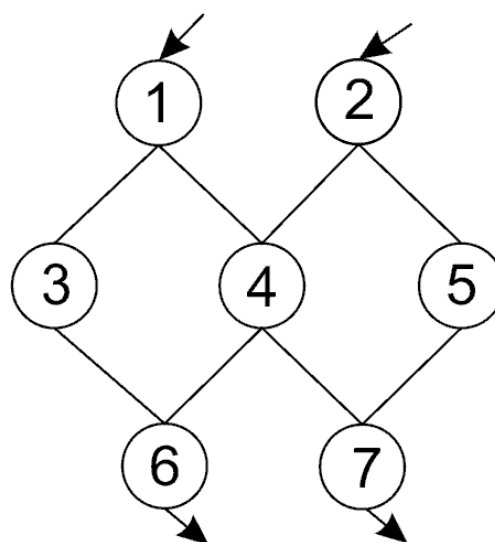


Схема № 12

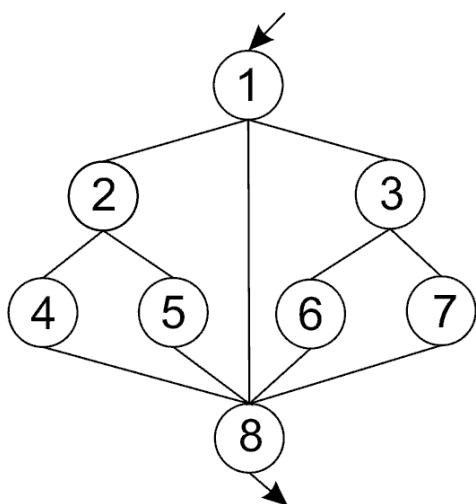


Схема № 13

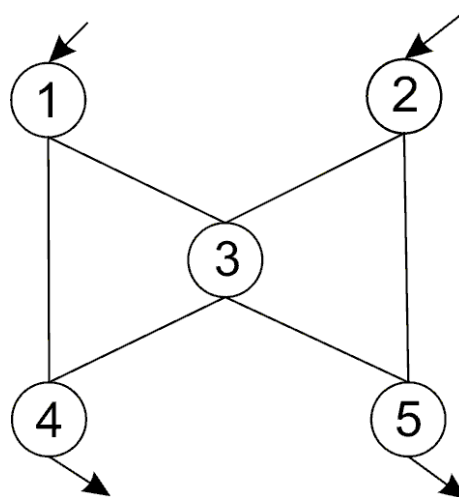


Схема № 14

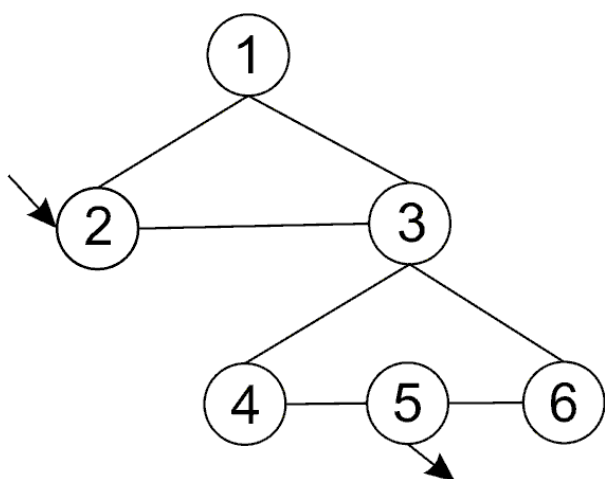


Схема № 15

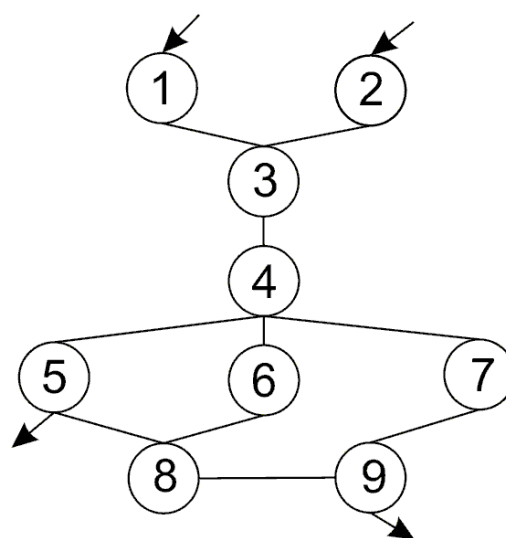


Схема № 16

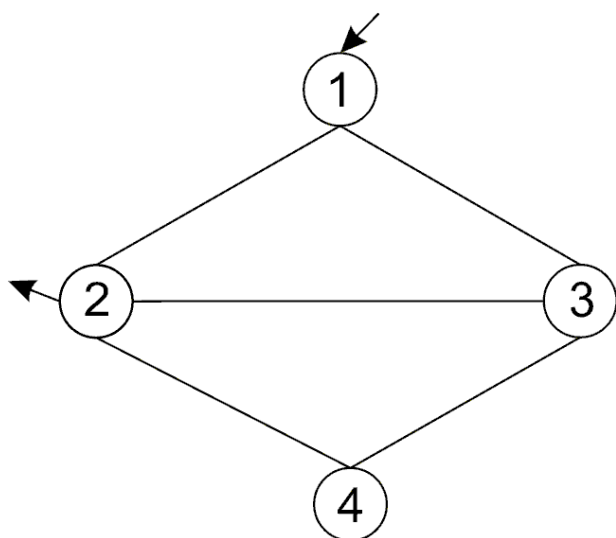


Схема № 17

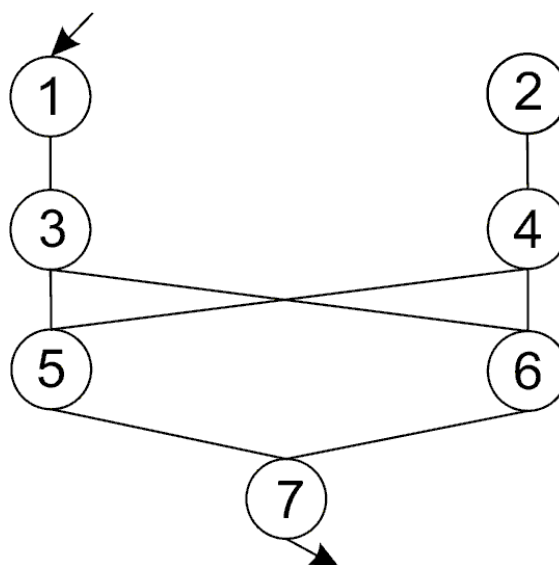


Схема № 18

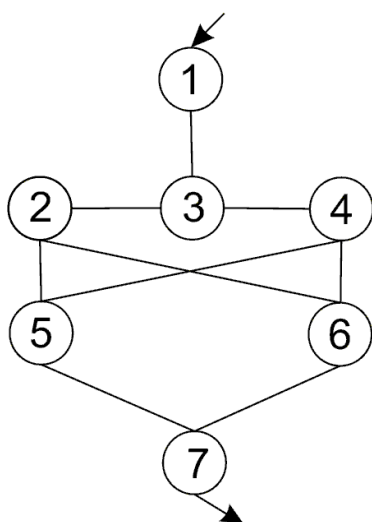


Схема № 19

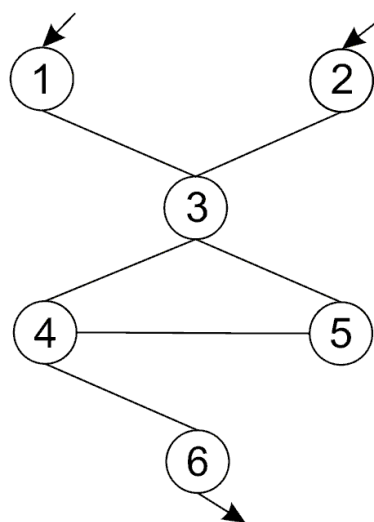


Схема № 20

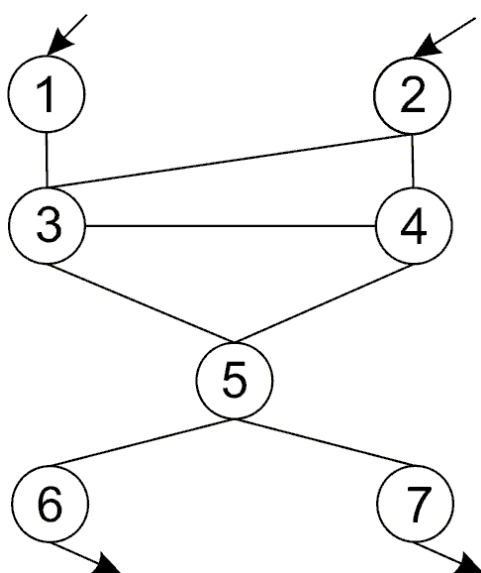


Схема № 21

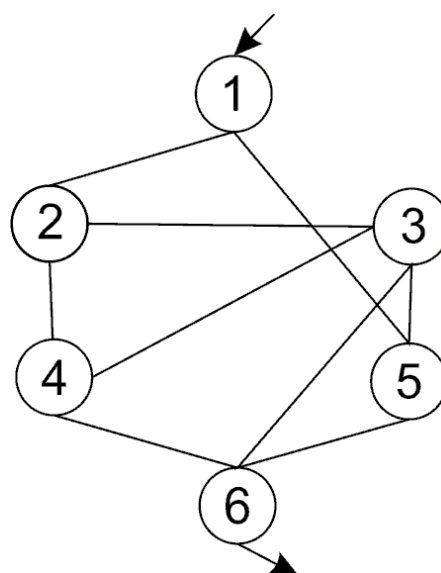


Схема № 22

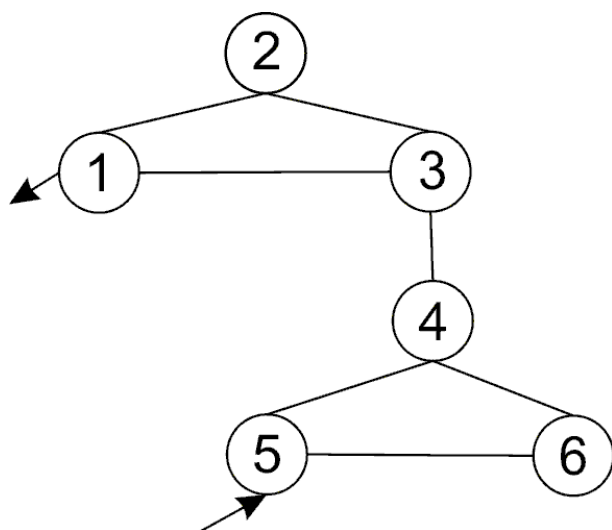


Схема № 23

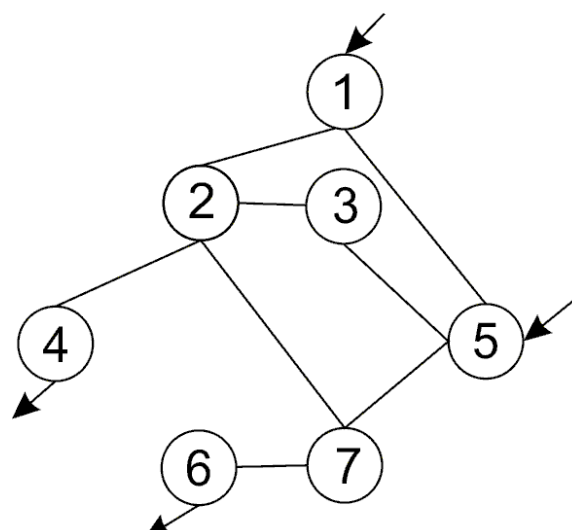


Схема № 24

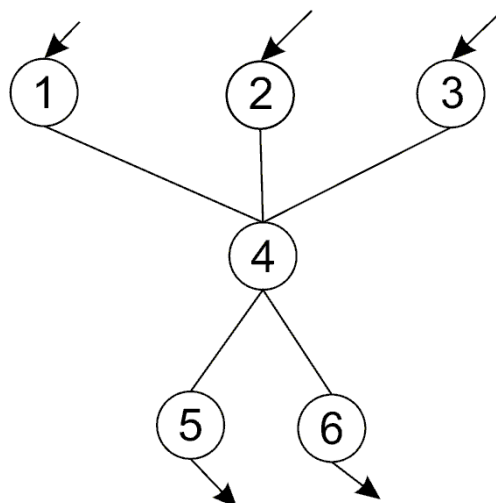


Схема № 25

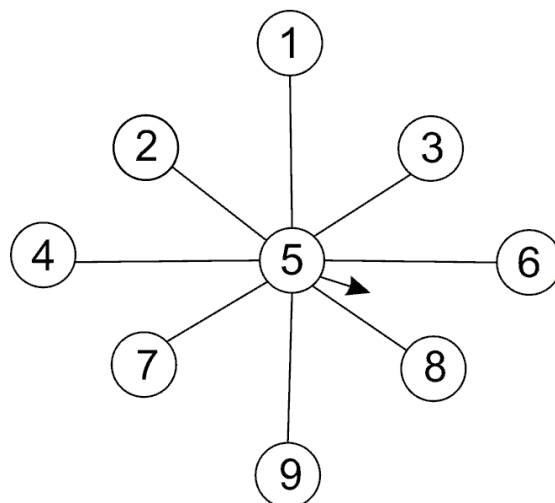


Схема № 26

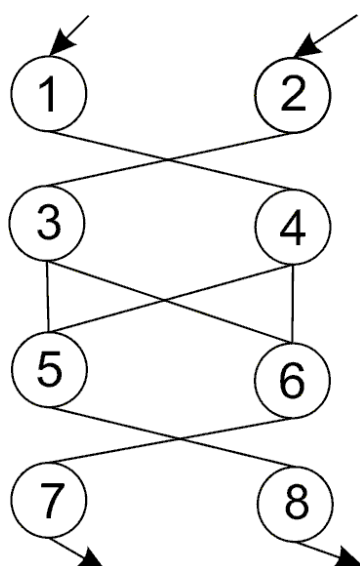


Схема № 27

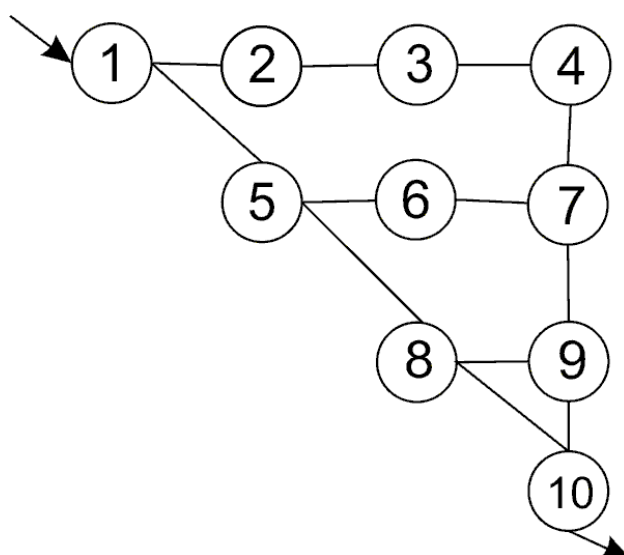


Схема № 28

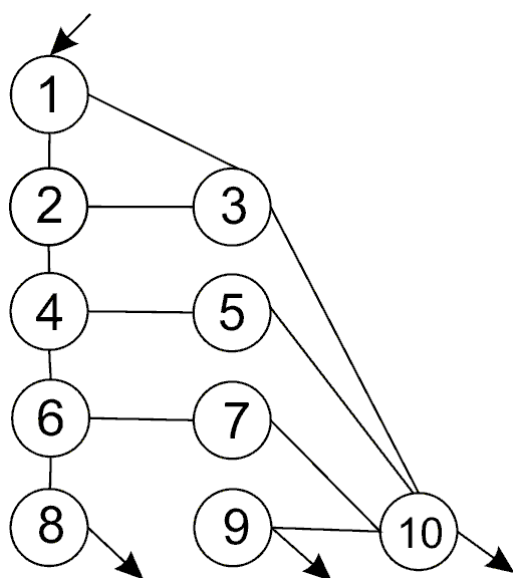


Схема № 29

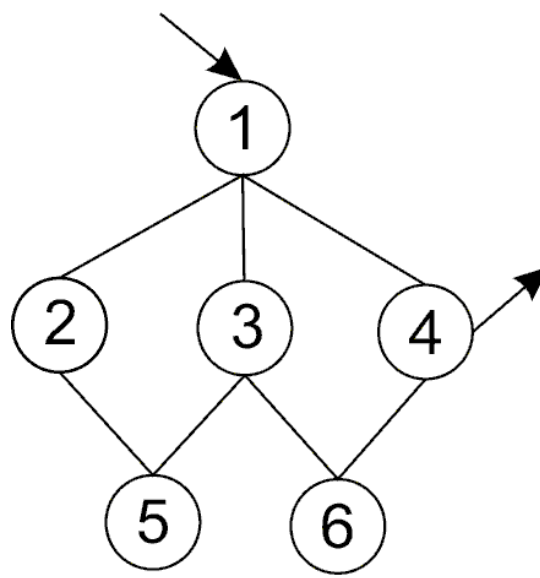


Схема № 30

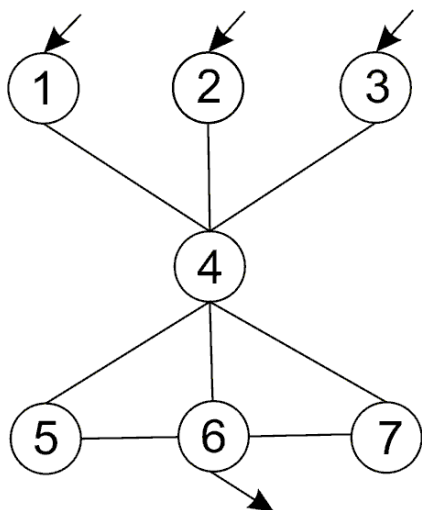


Схема № 31

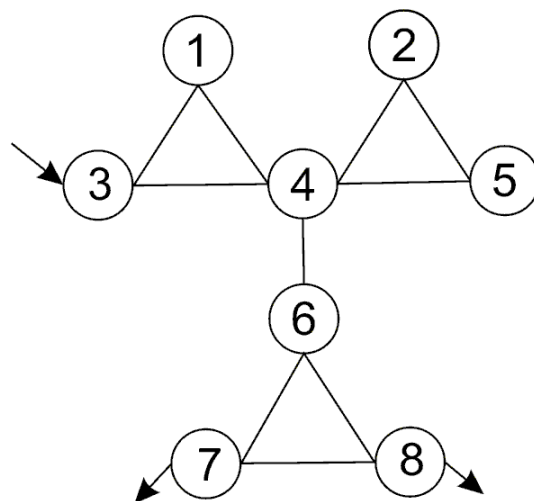


Схема № 32

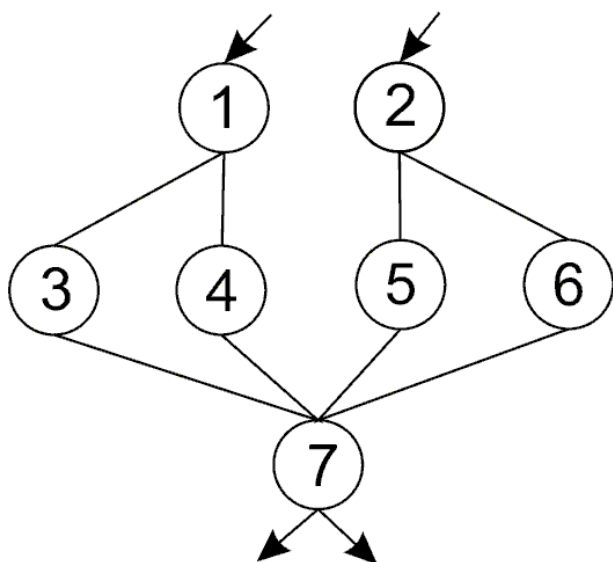


Схема № 33

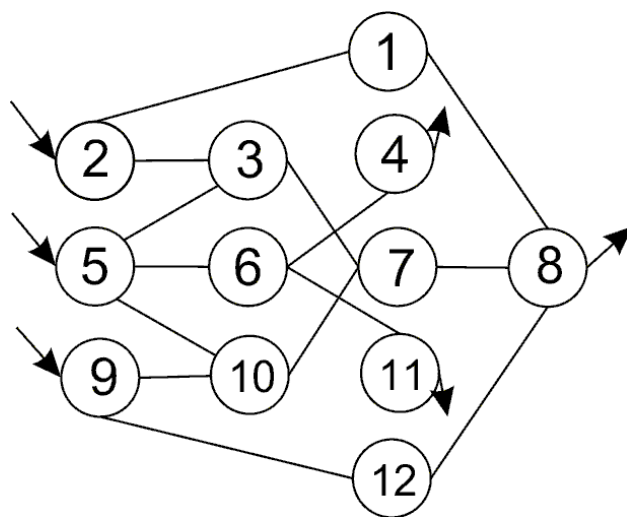


Схема № 34

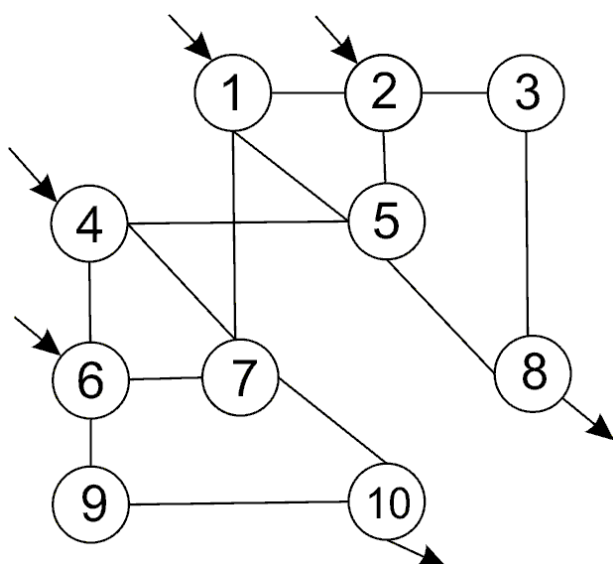


Схема № 35

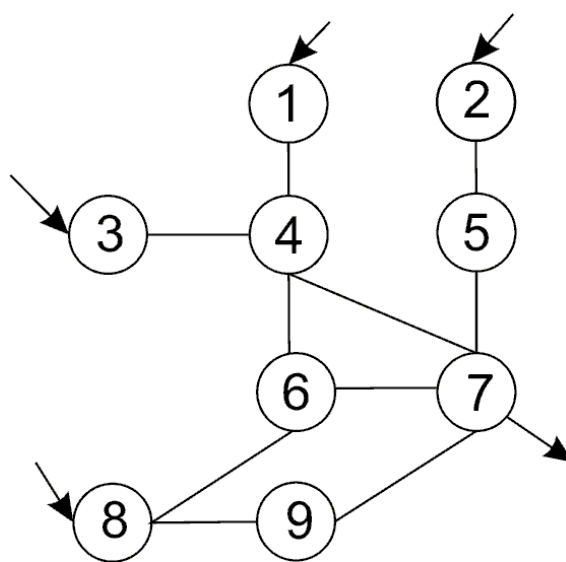


Схема № 36

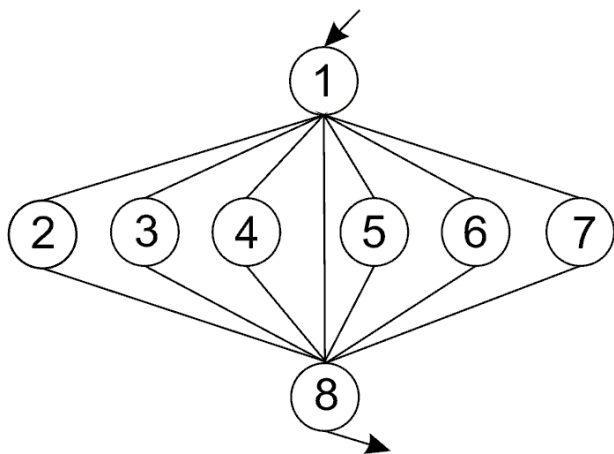


Схема № 37

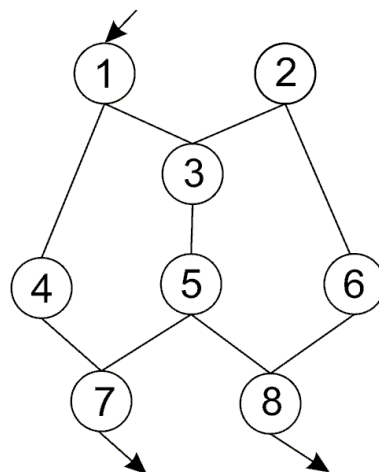


Схема № 38

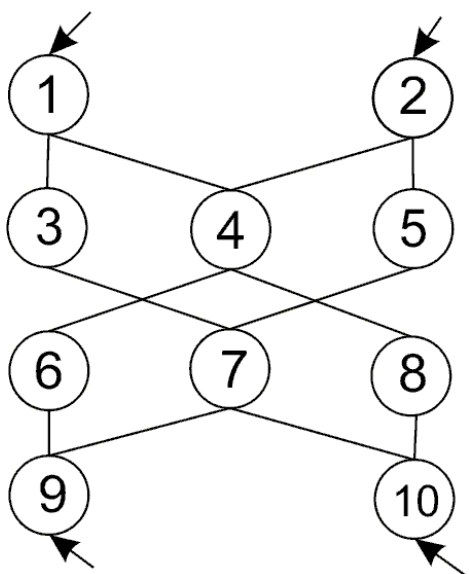


Схема № 39

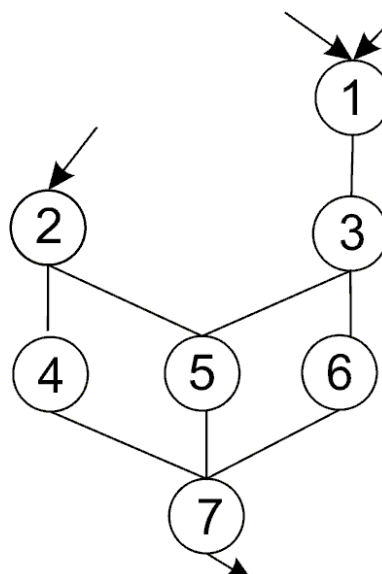


Схема № 40

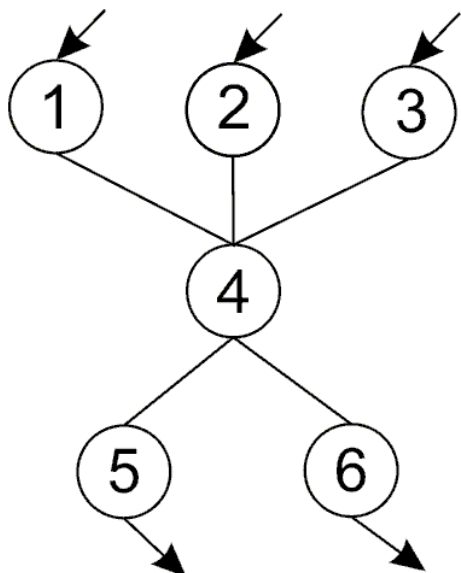


Схема № 41

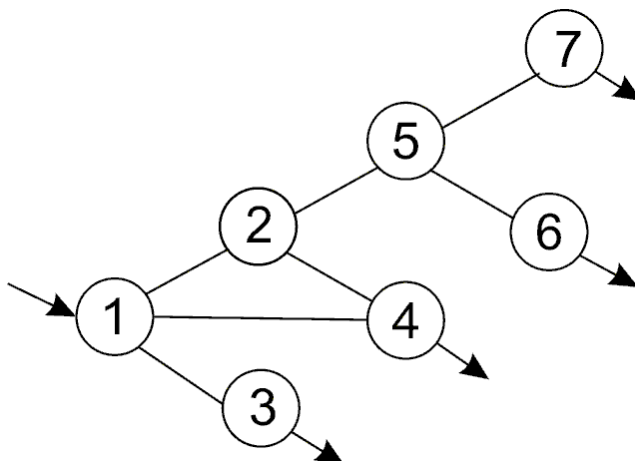


Схема № 42

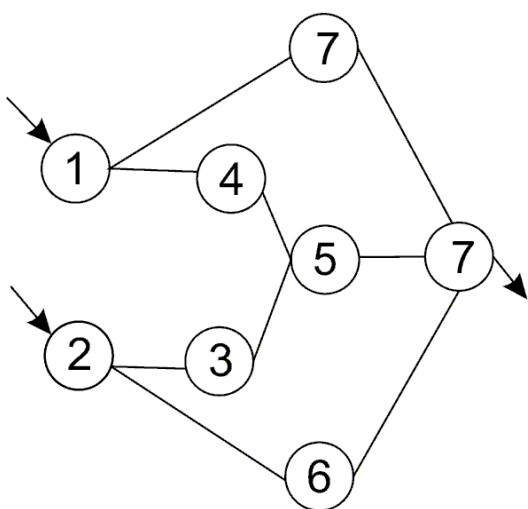


Схема № 43

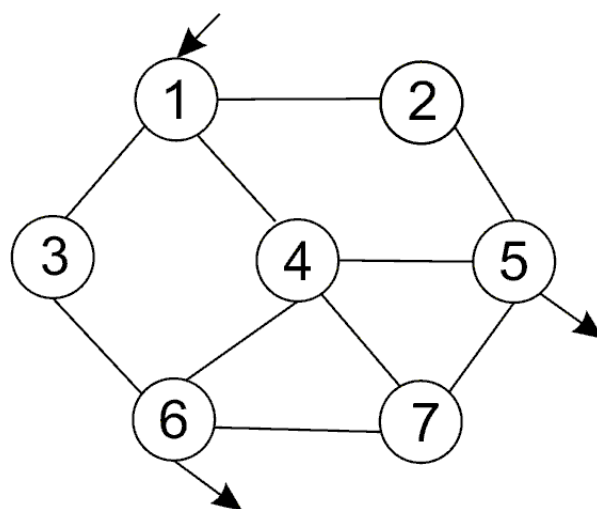


Схема № 44

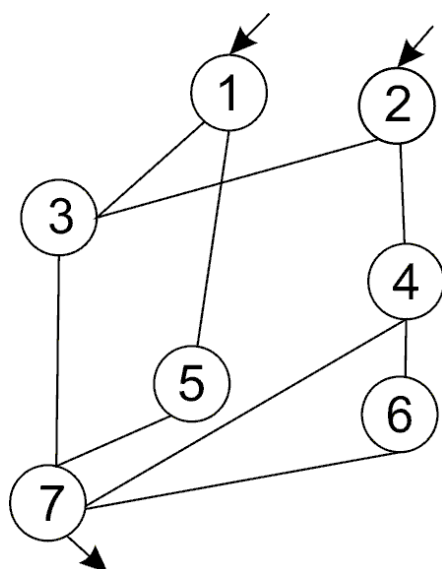


Схема № 45

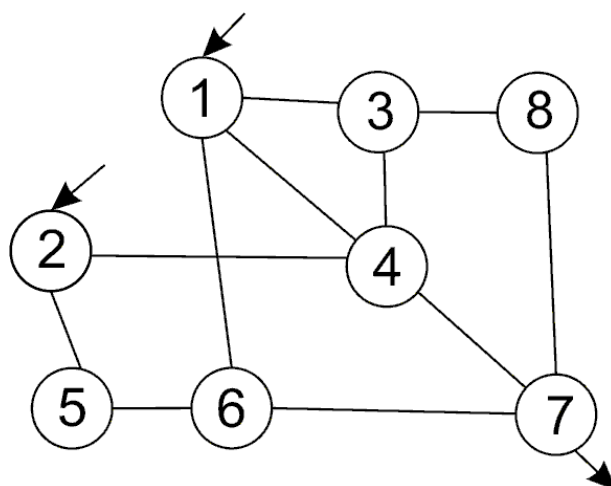


Схема № 46

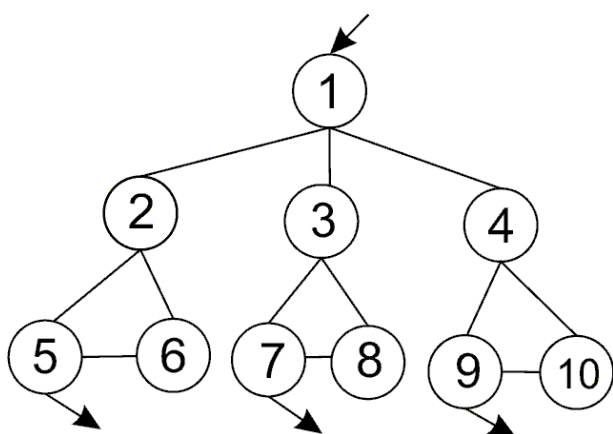


Схема № 47

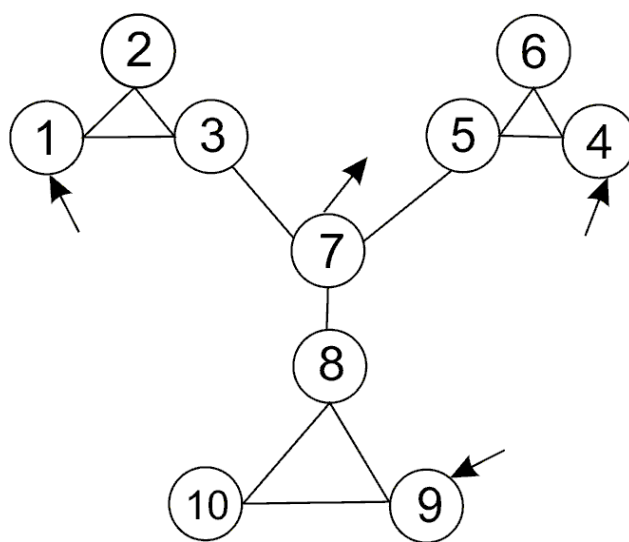


Схема № 48

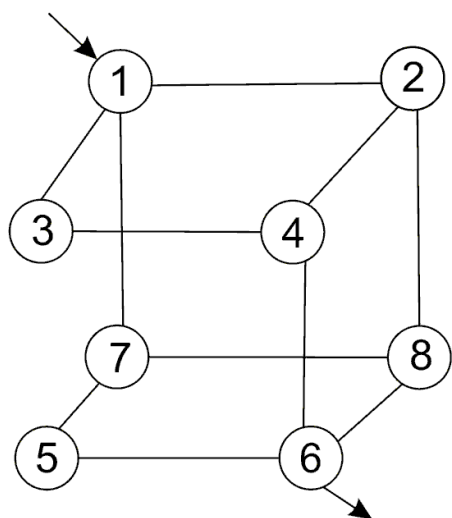


Схема № 49

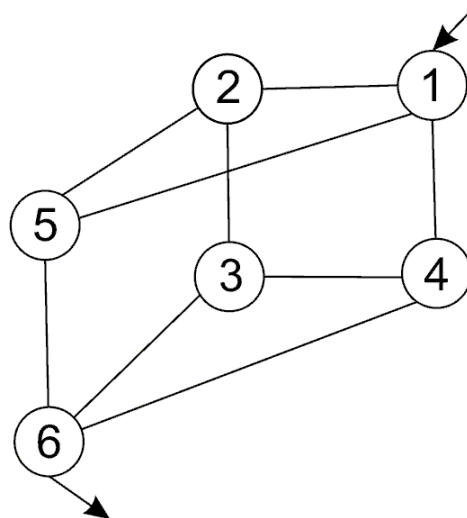


Схема № 50

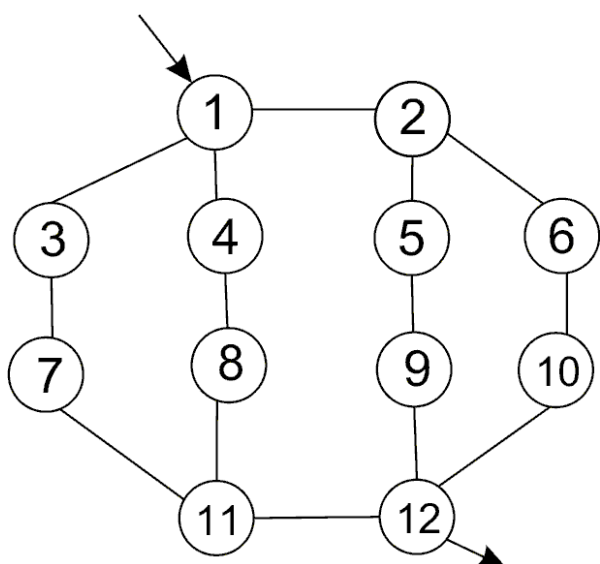


Схема № 51

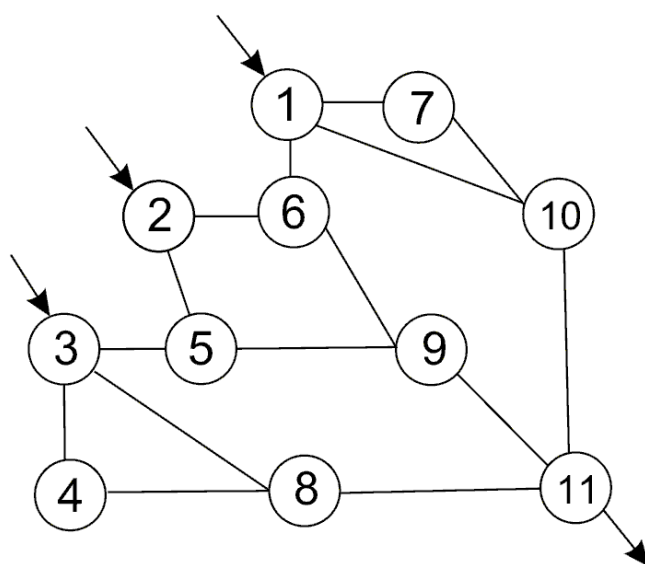


Схема № 52

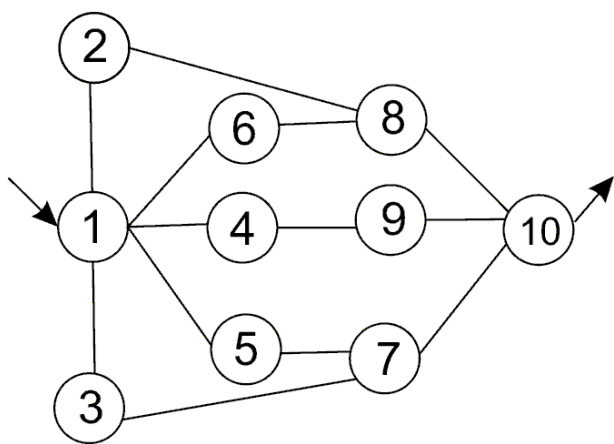


Схема № 53

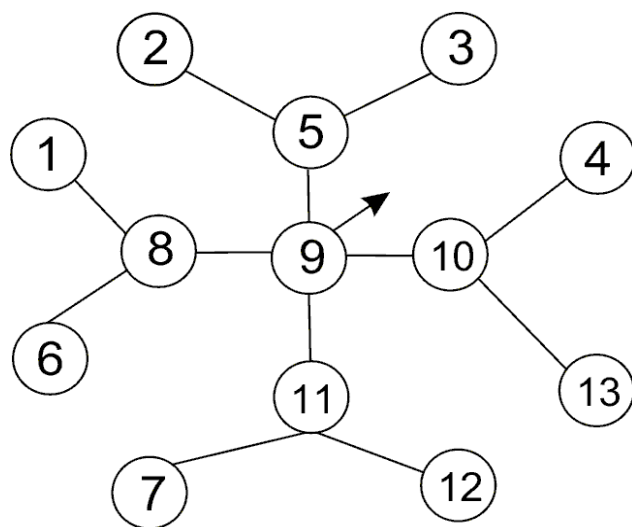


Схема № 54

ОГЛАВЛЕНИЕ

1. Лабораторная работа «Счетные ресурсы». Сравнение вычислительной мощности, предоставляемой операционными системами	3
2. Лабораторная работа «Память девичья». Сравнение объемов памяти предоставляемых, операционными системами	5
3. Лабораторная работа «Наследники». Создание дочерних процессов	7
4. Лабораторная работа «Трубы». Использование программных каналов	15
5. Лабораторная работа «На деревню дедушке». Использование очереди сообщений	20
6. Лабораторная работа «Вспомнить всё». Использование разделяемой памяти и семафоров	27
7. Лабораторная работа «Аргументы, параметры». Ассемблер из Си	40
8. Лабораторная работа «Поток сознания». Многопоточное программирование	45
9. Лабораторная работа «Мутанты среди нас». Использование мьютексов	53
10. Лабораторная работа «По радио объявили». Использование условных переменных	60
11. Лабораторная работа «Всё серьёзно». Не стандартные приемы работы с потоковыми примитивами синхронизации	66
12. Приложение 1. Варианты заданий	70
13. Приложение 2. Варианты схем	72